

Nhận dạng lại địa điểm và đóng vòng (Place Recognition and Loop Closure)

PHỤ THUỘC F/27-dat-hoc-sau-vao-dau (khung bốn cổng để quyết định thay hay không thay); F/28-dac-trung-va-ghep-cap (đặc trưng và ghép cặp học được — chương này dùng lại chúng ở tầng kiểm chứng); B/07-chia-du-lieu-va-danh-gia (giao thức đánh giá, recall và precision).

TÓM TẮT

Chương này mở đóng vòng thành bốn tầng, rồi dạy cách đo xem tầng nào đang giết vòng. Đọc xong bạn đặt được biển đếm đúng chỗ, dựng được nhãn đúng-sai từ chân trị, và phân biệt được “thiếu mẫu” với “trần inlier”. Bạn cũng biết khi nào một bộ mô tả toàn ảnh học được thật sự hơn túi từ, và khi nào nó chỉ dời nút thắt sang chỗ khác. Nếu chỉ nhớ một điều: người ta hay đổ lỗi cho tầng lấy ứng viên, nhưng trên một hệ stereo-inertial thì tầng kiểm chứng hình học — chỗ RANSAC ước lượng Sim3 — thường là bộ lọc mạnh hơn nhiều, và chương này dạy bạn cách tự đo xem điều đó có đúng với hệ của mình không thay vì tin vào một con số của người khác.

KIẾN THỨC CẦN CÓ TRƯỚC

- **đóng vòng (loop closure)** — việc nhận ra rằng robot đang ở lại một nơi đã đi qua, rồi thêm một ràng buộc nối hai thời điểm đó trong đồ thị tư thế. Đây là cơ chế duy nhất xoá được sai số tích lũy; không có nó thì mọi hệ đo bằng chuyển động tương đối đều trôi vô hạn.
- **ứng viên (candidate)** — một khung hình khoá cũ mà tầng truy hồi cho rằng có thể là cùng một nơi với khung hình hiện tại. Ứng viên chỉ là một đề xuất, chưa phải một vòng.
- **túi từ (bag of words)** — cách mô tả một ảnh bằng biểu đồ tần suất các “từ thị giác”, tức các cụm đặc trưng cục bộ đã được gom sẵn trong một từ điển. Hai ảnh giống nhau thì biểu đồ giống nhau, nên so ảnh trở thành so hai vectơ thưa.
- **mô tả toàn ảnh (global descriptor)** — một vectơ duy nhất, thường vài trăm tới vài nghìn chiều, tóm cả bức ảnh. Truy hồi địa điểm chính là tìm láng giềng gần nhất của vectơ này trong cơ sở dữ liệu.
- **Sim3** — phép biến đổi gồm xoay, tịnh tiến và một hệ số tỉ lệ chung, tức bảy bậc tự do. Bản đồ đơn ảnh có tỉ lệ trôi theo thời gian, nên hai đầu của một vòng thường lệch cả tỉ lệ chứ không chỉ lệch tư thế.
- **RANSAC và inlier** — RANSAC rút ngẫu nhiên một tập nhỏ nhất các khớp, dựng một giả thuyết từ đó, rồi đếm xem bao nhiêu khớp còn lại phù hợp với giả thuyết ấy. Các khớp phù hợp gọi là inlier; giả thuyết nhiều inlier nhất được giữ lại.
- **perceptual aliasing** — hiện tượng hai nơi khác nhau trông giống hệt nhau — hành lang lặp, dây cửa giống nhau, hàng cây đều. Đây là nguồn chính của vòng sai, và nó không giảm khi ta tăng độ phân giải ảnh.
- **recall và precision** — recall là tỉ lệ vòng có thật mà hệ phát hiện được; precision là tỉ lệ vòng được chấp nhận mà thật sự đúng. Hai số này đánh đổi nhau qua một ngưỡng, nên nói một số mà giấu số kia là vô nghĩa.

CÂU HỎI MỞ ĐẦU

- Q1. Hệ của bạn duyệt hàng chục nghìn ứng viên mà gần như không đóng được vòng nào. Bạn đặt bao nhiêu biến đếm, ở đúng những chỗ nào, để biết tầng nào đang giết vòng — và vì sao đặt sai một dòng làm bạn đổ tội nhầm cho suốt nhiều tháng?
- Q2. Nếu tầng kiểm chứng loại 99% ứng viên, đó là bằng chứng nó hỏng hay bằng chứng nó đang làm đúng việc? Bạn phân biệt hai khả năng đó bằng phép đo nào?
- Q3. Lý thuyết RANSAC nói rằng giảm tập mẫu tối thiểu từ ba điểm xuống hai điểm sẽ nâng xác suất hội tụ lên rất nhiều. Vì sao lời hứa đó vẫn có thể không đề ra thêm một vòng nào, và bạn kiểm giả định nào của nó trước khi bỏ công cài đặt?
- Q4. NetVLAD hơn túi từ trên mọi bảng xếp hạng nhận dạng địa điểm. Vì sao thay DBoW2 bằng nó vẫn có thể không đóng thêm được một vòng nào trên hệ của bạn?
- Q5. Vì sao điểm vận hành đúng của một bộ phát hiện vòng lại nằm lệch hẳn về phía precision, tới mức bạn sẵn sàng bỏ lỡ chín vòng để tránh một vòng sai?

1.1 Đặt vấn đề

Một hệ stereo-inertial đã tinh chỉnh kỹ vẫn trôi. Trôi là hệ quả tất yếu của việc cộng dồn các phép đo tương đối. Đóng vòng là cơ chế duy nhất xoá được phần trôi đã tích lũy, vì nó là ràng buộc duy nhất nối hai thời điểm cách xa nhau. Vì vậy khi vòng không nổ, sai số cuối chuỗi không có gì kéo về.

Câu hỏi mà hầu hết mọi người đặt ở điểm này là câu hỏi sai. Họ hỏi: “bộ nhận dạng địa điểm của tôi yếu, có nên thay túi từ bằng một mạng học được không?”. Câu hỏi ấy giả định trước rằng nút thất nằm ở tầng truy hồi. Giả định đó thường sai, và nó sai theo một cách rất tốn kém: bạn bỏ ba tháng thay một tầng vốn không phải thủ phạm.

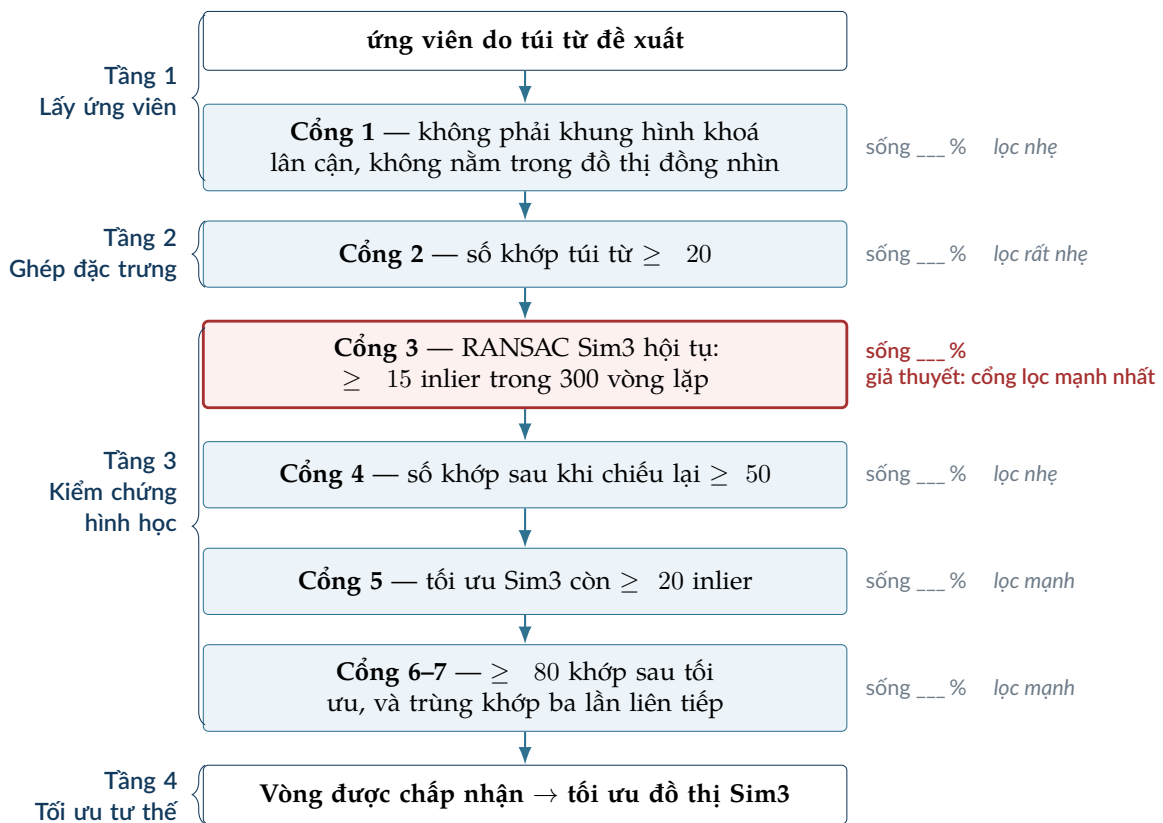
Đóng vòng không phải một khối. Nó là một dây chuyền bốn tầng, mỗi tầng là một bộ lọc riêng. Tầng lấy ứng viên đề xuất các khung hình khoá cũ có thể trùng nơi. Tầng ghép đặc trưng dựng các cặp điểm tương ứng giữa hai khung hình. Tầng kiểm chứng hình học hỏi xem tập cặp đó có giải thích được bằng một phép biến đổi cứng duy nhất không. Tầng tối ưu nhận ràng buộc đã được chấp nhận và phân bổ lại sai số cho cả bản đồ.

Mỗi tầng loại bỏ một phần ứng viên. Tổng của bốn tầng là một cái phễu rất hẹp. Nếu bạn chỉ nhìn đầu ra của phễu, bạn thấy đúng một con số: số vòng đóng được. Con số đó không cho biết tầng nào hẹp. Giá trị của chương này là biến câu “vòng không nổ” thành bốn con số riêng, mỗi con số cho một tầng, đo được trong một buổi chiều.

Sau khi có bốn con số đó, chương mới trả lời câu hỏi ban đầu. Bộ mô tả toàn ảnh học được giải quyết việc gì mà túi từ không giải quyết được, và giải quyết ở tầng nào. Định vị lại bằng mạng nơ-ron đứng ở đâu trong bốn tầng ấy. Ghép cặp học được đưa vào tầng kiểm chứng thì đổi gì. Và cuối cùng, làm sao đo cho đúng, khi mà một vòng sai đắt hơn nhiều lần một vòng bị bỏ lỡ.

Phần F bám bốn nỗi đau của một hệ đã chạy thật: chuyển động nhanh, ảnh mờ do chuyển động, đóng vòng, và đặt trọng số cho IMU. Chương này chạm thẳng vào nỗi đau thứ ba, và chỉ chạm gián tiếp vào hai nỗi đau đầu. Chuyển động nhanh và ảnh mờ xuất hiện ở đây dưới dạng hệ quả: chúng làm chất lượng cặp tương ứng tụt, và chính chất lượng cặp là thứ quyết định tầng ba. Trọng số IMU không thuộc chương này, trừ một chỗ: phương trọng lực từ IMU được dùng để rút bậc tự do trong bài toán kiểm chứng.

Hình 1.1 là bản đồ của cả chương. Nó vẽ bốn tầng cùng chuỗi công cụ thể của một hệ kiểu ORB-SLAM3, và chỗ để bạn tự điền tỉ lệ sống sót đo được trên hệ của mình. Đọc nó trước, rồi mỗi mục sau chỉ là một trạm trên đúng cái phễu đó.



Hình 1.1. Bốn tầng của đóng vòng và chuỗi cổng cụ thể thực hiện chúng; các ngưỡng ghi trong ô là hằng số trong mã nguồn của một hệ kiểu ORB-SLAM3, không phải kết quả đo. Các ô “sống ___ %” là chỗ trống bạn điền sau khi chạy phép đo ở mục C: mỗi tỉ lệ là *có điều kiện*, tức phần trăm ứng viên tới được cổng đó mà vượt qua nó. Nhân định tính bên phải là giả thuyết làm việc, không phải số đo: cổng kiểm chứng hình học thường là chỗ lọc mạnh nhất trong bốn tầng, và trên nhiều hệ thống nó cũng là nơi phần lớn ứng viên đúng bị loại. Chương này không nói tỉ lệ ấy bằng bao nhiêu trên hệ của bạn — nó dạy bạn cách tự lấy con số đó, và mục D dạy tiếp cách biết tỉ lệ ấy là loại đúng hay loại sai.

TRỰC GIÁC

Hình dung một quy trình tuyển dụng bốn vòng. Cuối quý bạn thấy phần lớn ứng viên bị loại ở vòng phỏng vấn kỹ thuật. Phản xạ đầu tiên là sửa vòng phỏng vấn: nói thang điểm, đổi người hỏi. Nhưng nếu bạn mở lại hồ sơ những người bị loại, gần như tất cả họ thật sự không đạt. Vòng phỏng vấn đang làm đúng việc của nó. Vài người giỏi bị loại oan thì bị loại vì hồ sơ gửi lên quá sơ sài, nên người phỏng vấn không có gì để hỏi. Chỗ phải sửa là khâu chuẩn bị hồ sơ, ở phía trước. Tầng kiểm chứng hình học trong đóng vòng đúng đúng vị trí vòng phỏng vấn ấy.

1.2 Bốn tầng và cách đo tầng nào đang giết vòng (Anatomy and Instrumentation)

A. Mỗi tầng làm gì, và giả định ngầm của nó

Tầng một, lấy ứng viên, so một mô tả toàn ảnh với cơ sở dữ liệu và trả về danh sách ngắn. Giả định ngầm là hai ảnh cùng một nơi thì gần nhau trong không gian mô tả. Giả định này gây khi ánh sáng đổi, khi góc nhìn lệch lớn, và khi cảnh có cấu trúc lặp.

Tầng hai, ghép đặc trưng, dựng các cặp điểm tương ứng giữa khung hình hiện tại và ứng viên. Trong ORB-SLAM3, việc này được dẫn hướng bằng chính cây từ điển: chỉ so các đặc trưng rơi

vào cùng một nút từ vựng [1]. Giả định ngầm là hai đặc trưng của cùng một điểm ba chiều sẽ rơi vào cùng một nút. Đây là một giả định mạnh, và nó là chỗ số lượng cặp bị chặn trên.

Tầng ba, kiểm chứng hình học, hỏi một câu duy nhất: có tồn tại một phép Sim3 giải thích được đủ nhiều cặp không. Đây là câu hỏi có đáp án đúng hoặc sai, nên nó là tầng duy nhất trong bốn tầng thật sự *chứng minh* được điều gì. Giả định ngầm là tỉ lệ inlier đủ cao để RANSAC bắt trúng một mẫu sạch.

Tầng bốn, tối ưu, đưa ràng buộc mới vào đồ thị và phân bổ lại sai số. Giả định ngầm ở đây là ràng buộc mới đúng. Nếu nó sai, tầng bốn không phát hiện được; nó chỉ trung thành lan cái sai ra khắp bản đồ.

B. Hai cổng rẻ nhất: ràng buộc thời gian và đồng nhìn

Trước khi bàn tới bất cứ mô-đun học được nào, cần thấy hai cổng rẻ nhất trong cả chuỗi. Cả hai không tốn một phép nhân nào, và cả hai mua precision bằng cách trả recall.

Cổng thứ nhất loại các khung hình khoá đang nằm trong vùng đồng nhìn hiện tại. Không có nó, hệ sẽ “đóng vòng” với chính chỗ nó vừa đi qua ba giây trước. Ràng buộc kiểu đó đúng nhưng vô ích, vì nó không mang thông tin mới cho đồ thị. Trong bảng sống sót, đây chính là cổng một.

Cổng thứ hai là ràng buộc trùng khớp nhiều lần liên tiếp. ORB-SLAM3 chỉ chấp nhận một vòng khi ba khung hình khoá liên tiếp cùng chỉ về một nhóm đồng nhìn. Cơ chế đằng sau rất gọn: perceptual aliasing thường không bền theo thời gian. Hai hành lang giống nhau cho điểm cao ở một khung hình, nhưng hiếm khi giữ được điểm cao qua ba khung hình liên tiếp với hình học nhất quán.

Giá phải trả thì cụ thể và đo được. Bạn mất ba khung hình khoá đầu của mỗi lần đi lại. Với một lần đi lại rất ngắn, chẳng hạn robot chỉ lướt qua chỗ cũ trong hai giây, bạn mất luôn cả vòng. Đây là ví dụ sạch nhất trong toàn chương về việc đổi recall lấy precision, và nó xảy ra trước khi bất kỳ mạng nào được gọi.

Có một biến thể đáng biết. Thay vì chấm điểm từng khung hình khoá riêng lẻ, ta gom các khung hình khoá cùng nhóm đồng nhìn lại rồi cộng điểm của cả nhóm [2]. Cách này giảm nhiễu của điểm số truy hồi, vì một nơi đúng thường làm nhiều khung hình khoá lân cận cùng ăn điểm. Đó là một dạng kiểm chứng không gian rất rẻ, làm ngay tại tầng một.

C. Đặt biến đếm ở đâu, và cái bẫy dịch nhãn

Cách đo rất rẻ. Với mỗi ứng viên, tăng một biến đếm ngay trước mỗi cổng, rồi ghi ra một dòng nhật ký cho mỗi khung hình khoá. Sau vài chục lần chạy bạn có bảng sống sót từng tầng. Toàn bộ công việc là vài chục dòng mã và không ảnh hưởng thời gian thực.

Nhưng có một cái bẫy, và nó đã làm hỏng nhiều tuần phân tích trong chính dự án này.

CẠM BÃY

Nếu bạn đặt lệnh tăng biến đếm *ngay trước* câu lệnh `if` của cổng, thì cột mang tên cổng đó thật ra đang đếm “số ca đã vượt cổng *trước* nó”. Mọi nhãn cột bị dịch đi một bậc. Hậu quả rất cụ thể và rất khó thấy: cái cổng bị đổ tội luôn là cổng đứng *sau* thủ phạm thật, và vì bảng vẫn nhất quán với chính nó nên không có triệu chứng nào để bạn nghi ngờ. Trong chính dự án này, một bảng đặt sai kiểu đó đã dẫn phân tích đi lạc suốt nhiều tuần trước khi được đếm lại từ nhật ký thô. Cách phòng: đặt lệnh tăng biến đếm *bên trong* khối `if`, hoặc viết ngay trong tên biến rằng nó đếm “đã tới cổng nào” chứ không phải “đã qua cổng nào”. Bài học tổng quát: một cột nhật ký không tự nói ra ngữ nghĩa của nó, nên ngữ nghĩa phải nằm trong tên.

D. Nhãn đúng-sai từ chân trị: phép đo quan trọng thứ hai

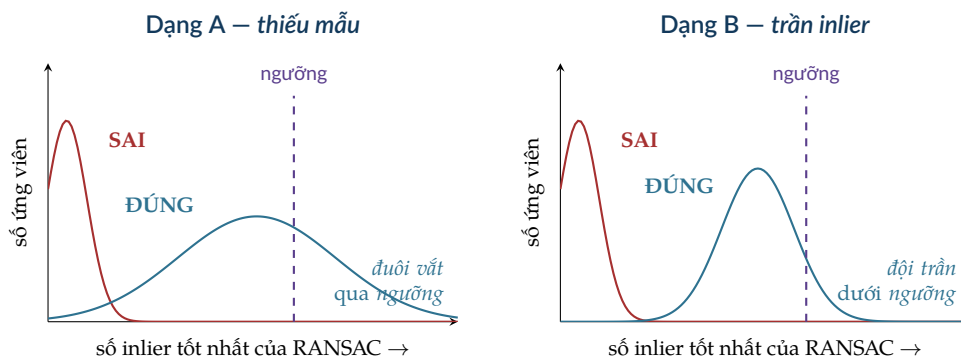
Bảng sống sót nói tăng nào loại nhiều nhất. Nó không nói tăng đó loại *đúng* hay *sai*. Đây là phân biệt quan trọng nhất của cả chương, và nó cần một phép đo thứ hai.

Phép đo ấy là gắn nhãn cho từng ứng viên bằng chân trị. Với mỗi cặp khung hình khoá được đề xuất, tra tư thế thật của cả hai, tính khoảng cách giữa chúng, rồi gọi ứng viên là “đúng” nếu khoảng cách dưới một ngưỡng và “sai” nếu vượt xa ngưỡng đó. Ngưỡng ấy là một lựa chọn thiết kế chứ không phải sự thật: trong nhà, vài mét là mức hợp lý; ngoài trời thì phải rộng hơn nhiều. Khi đã có nhãn, bảng sống sót tách làm hai bảng, và hai bảng đó kể hai câu chuyện khác nhau.

Việc phải làm sau đó là vẽ, cho riêng từng nhóm nhãn, phân bố của số *inlier* tốt nhất mà RANSAC tìm được. Đây là biểu đồ quan trọng nhất của cả chương, vì hình dạng của nó trả lời thẳng câu hỏi “cổng này loại đúng hay loại sai”. Có hai câu hỏi đọc được từ nó, và cả hai đều đọc bằng mắt.

1. **Các ứng viên sai có bao giờ tới gần ngưỡng không?** Nếu phân bố của chúng nằm sát số không và tắt hẳn từ lâu trước ngưỡng, thì mọi ca chết trong nhóm đó là *loại đúng*, và bạn không có lý do gì để dưng vào cổng. Một tỉ lệ sống sót rất thấp khi ấy là bằng chứng cổng đang làm đúng việc, không phải bằng chứng nó hỏng.
2. **Giữa hai phân bố có một khoảng trống không, và ngưỡng nằm ở đâu so với khoảng trống ấy?** Nếu có một khoảng mà chỉ ứng viên *đúng* chết trong đó và không ứng viên sai nào chạm tới, thì đúng chỗ đó — và chỉ chỗ đó — là dư địa mà việc nói ngưỡng lấy lại được. Đếm số ứng viên nằm trong khoảng trống ấy là bạn có ngay trần lợi ích của mọi ý tưởng “nói ngưỡng”, trước khi cài đặt gì.

Hình 1.2 vẽ hai hình dạng mà biểu đồ ấy có thể mang, và chúng dẫn tới hai kết luận trái ngược nhau. Điểm cốt lõi: một tỉ lệ sống sót thấp ở cổng ba *không* tự nó chứng minh điều gì. Cùng một tỉ lệ ấy có thể là dấu hiệu của một bộ kiểm chứng khoẻ mạnh hoặc của một tăng ghép cặp đôi cặp đúng, và chỉ hình dạng phân bố mới phân biệt được.



Hình 1.2. Hai hình dạng mà biểu đồ số inlier tốt nhất có thể mang, vẽ theo *cơ chế* chứ không theo số đo — các trục cố ý không có vạch số, vì con số là thứ bạn phải tự lấy trên hệ của mình. Ở cả hai hình, nhóm ứng viên SAI nằm sát số không và tắt trước ngưỡng, nghĩa là mọi ca chết trong nhóm ấy là loại đúng và cổng đang làm việc của nó. Khác biệt nằm ở nhóm ĐÚNG. Dạng A: phân bố trải rộng và vượt qua ngưỡng, tức inlier vốn có đủ mà chỉ thiếu lượt rút trúng — đây là *thiếu mẫu*, và tăng số vòng lặp hay rút mẫu có ưu tiên sẽ có lãi. Dạng B: phân bố đội trần ngay dưới ngưỡng và không có đuôi, tức số cặp đúng thật sự tồn tại đã ít hơn ngưỡng — đây là *trần inlier*, và không cách rút mẫu nào chạm tới được. Nhìn nhầm dạng B thành dạng A là cách tốn nhiều tháng nhất để không sửa được gì.

E. Phân biệt “thiếu mẫu” với “trần inlier”

Khi RANSAC không hội tụ, có đúng hai nguyên nhân, và chúng đòi hai cách chữa trái ngược nhau.

- **Thiếu mẫu (sampling failure)** — inlier có đủ, nhưng số vòng lặp không đủ để rút trúng một tập mẫu tối thiểu toàn inlier. Cách chữa: tăng số vòng lặp, giảm cỡ tập mẫu tối thiểu, hoặc rút mẫu có ưu tiên.
- **Trần inlier (inlier ceiling)** — số cặp đúng thật sự tồn tại đã ít hơn ngưỡng. Không cách rút mẫu nào cứu được. Cách chữa duy nhất là tạo ra nhiều cặp đúng hơn, tức sửa tăng hai.

Hai nguyên nhân này phân biệt được bằng một biểu đồ duy nhất, chính là Hình 1.2. Nếu số inlier tốt nhất của các ứng viên đúng phân bố cao nhưng thưa thớt qua ngưỡng, đó là thiếu mẫu. Nếu nó đội trần ngay dưới ngưỡng, đó là trần inlier.

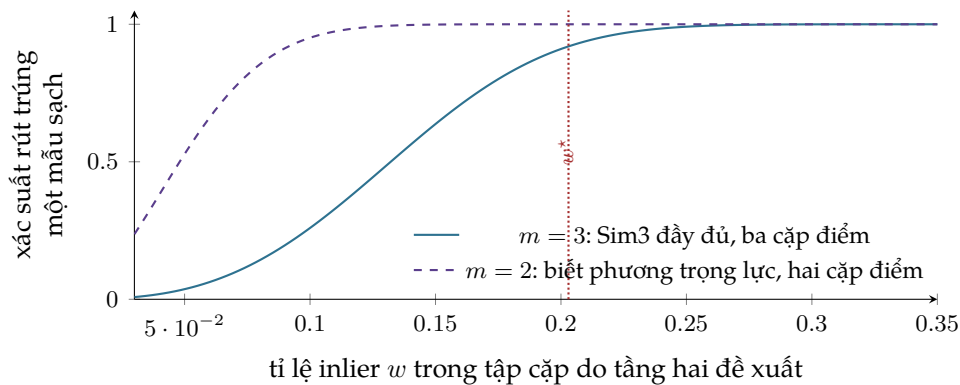
Có một con số thứ hai phải ghi kèm để đọc biểu đồ ấy cho đúng, và nó là mẫu số: *số cặp mà tăng hai đề xuất*. Số inlier tốt nhất chỉ có nghĩa khi so với mẫu số đó, vì tỉ lệ inlier w mới là đại lượng đi vào công thức bên dưới. Mười lăm inlier trên hai trăm cặp và mười lăm inlier trên bốn mươi cặp là hai tình huống hoàn toàn khác nhau. Ghi cả tử lẫn mẫu vào cùng một dòng nhật ký là việc rẻ nhất bạn làm được, và không ghi nó là lý do phổ biến nhất khiến bảng của bạn không trả lời được câu hỏi nào.

Lý thuyết RANSAC làm rõ tại sao phân biệt này quan trọng. Xác suất rút được ít nhất một tập mẫu sạch trong N vòng lặp là

$$p = 1 - (1 - w^m)^N,$$

với w là tỉ lệ inlier và m là cỡ tập mẫu tối thiểu [3]. Với Sim3 tổng quát thì $m = 3$, vì ba cặp điểm ba chiều đủ để giải bài toán định hướng tuyệt đối có tỉ lệ [4], [5]. Đây là một đẳng thức, nên thay số vào là ra ngay, và việc thay số cho thấy quan hệ dốc tới mức nào. Giả sử $N = 300$ vòng lặp: ở $w = 0,20$ ta được $p \approx 0,91$, còn ở $w = 0,10$ chỉ còn $p \approx 0,26$. Hai giá trị w này là giả định để thấy hình dạng quan hệ, không phải số đo của hệ nào; điều đáng nhớ là chia đôi tỉ lệ inlier đủ để biến một cổng gần như luôn mở thành một cổng gần như luôn đóng.

Nhưng nó chỉ là nút thắt nếu nguyên nhân là thiếu mẫu. Hình 1.3 vẽ hai đường cong cho $m = 3$ và $m = 2$. Khoảng cách giữa chúng ở vùng w thấp là rất lớn, nên lý thuyết hứa hẹn một cải thiện mạnh nếu ta giảm được cỡ tập mẫu. Mục 1.5 mổ ba giả định ẩn của lời hứa đó, và nói cách kiểm từng giả định trước khi bỏ công cài đặt.



Hình 1.3. Đồ thị của đúng công thức $1 - (1 - w^m)^N$ với $N = 300$; đây là toán, không phải số đo. Giảm cỡ tập mẫu tối thiểu từ ba xuống hai đẩy cả đường cong sang trái rất mạnh ở vùng tỉ lệ inlier thấp — đó là toàn bộ sức hấp dẫn của ý tưởng dùng trọng lực để rút bậc tự do. Đường chấm đứng là w^* , tỉ lệ inlier tối thiểu để chạm ngưỡng, tính bằng ngưỡng inlier chia cho số cặp mà tăng hai đề xuất; bạn tự tính nó cho hệ mình từ hai cột nhật ký ở Bảng 1.4. Bên trái w^* thì hai đường cong này vô nghĩa, vì ở đó số inlier *thật* đã dưới ngưỡng và không cách rút mẫu nào chạm tới được.

F. Ba con số nên có trước khi đọc bất cứ paper nào

Trước khi cân nhắc thay một tăng, hãy có ba con số cho hệ của chính bạn.

1. **Bảng sống sót từng cổng**, với biến đếm đặt đúng chỗ. Nó cho biết tầng nào hẹp.
2. **Bảng sống sót tách theo nhãn chân trị**. Nó cho biết tầng hẹp đó loại đúng hay loại sai.
3. **Số ứng viên đúng tồn tại**. Nếu một chuỗi không có lần đi lại nào thì việc nó không đóng vòng là kết quả đúng, không phải lỗi.

Con số thứ ba hay bị bỏ sót nhất, và nó rẻ đến mức không có lý do gì để bỏ. Với mỗi chuỗi, quét mọi cặp khung hình khoá bằng *chân trị* rồi đếm xem có bao nhiêu cặp thoả điều kiện “cùng nơi” mà không phải khung hình khoá lân cận về thời gian. Nếu con số đó bằng không thì chuỗi ấy đơn giản là không có lần đi lại nào, và việc hệ không đóng vòng trên nó là kết quả *đúng*, không phải lỗi. Nhiều chuỗi phổ biến trong nhà là đường đi một chiều không quay lại, nên khả năng bạn đang chữa một hệ vốn đang đúng cao hơn nhiều so với cảm giác. Không đo con số này trước là cách chắc chắn nhất để mất nhiều tuần.

↔ **cùng nguyên lý với E/24** — đây chính là thí nghiệm oracle: thay tầng lấy ứng viên bằng chân trị để đo trần đóng góp của nó, trước khi tối ưu nó

1.3 Toàn ảnh học được: từ túi từ tới SALAD và AnyLoc (Learned Global Descriptors)

A. DBoW2 làm được gì, và nó gãy ở đâu

Túi từ nhị phân của DBoW2 là một thiết kế rất kinh tế [2]. Nó dùng lại đúng các mô tả ORB đã được trích cho việc bám vết, nên chi phí trích đặc trưng bằng không. Từ điển được huấn luyện ngoại tuyến một lần, dạng cây phân cấp với khoảng một triệu từ. Truy vấn là tra chỉ mục ngược trên vectơ thừa, nên rẻ.

Về chi phí, điều đáng nói không phải một con số mà là một tính chất: chi phí biên của tăng một trong một hệ kiểu ORB-SLAM3 gần như bằng không, vì mọi thứ nó cần — các mô tả ORB — đã được trích sẵn cho việc bám vết. Trước khi tin điều đó trên hệ của mình, hãy đo hai thứ: độ trễ của tầng phát hiện ở *phân vị* 99, không phải trung vị, và độ dài hàng đợi khung hình khoá. Nếu hàng đợi luôn rỗng thì tầng một chưa phải chỗ đáng động tới, và mọi lập luận về việc thay nó phải dựa trên chất lượng chứ không dựa trên tốc độ.

DBoW2 gãy ở bốn chỗ, và cả bốn đều bắt nguồn từ cùng một điểm: nó thừa hưởng giới hạn của chính mô tả ORB.

- **Đổi ánh sáng mạnh** — ngày với đêm, trong nhà bật tắt đèn. Mô tả nhị phân dựa trên so sánh cường độ điểm ảnh, nên đảo tương phản làm chúng đổi hẳn.
- **Lệch góc nhìn lớn** — đi lại cùng hành lang theo chiều ngược, hoặc lệch ngang vài mét. Đây là chế độ hỏng thường gặp nhất trong nhà.
- **Cấu trúc lặp** — perceptual aliasing. Túi từ bỏ hoàn toàn thông tin bố cục không gian, nên hai hành lang giống nhau cho cùng biểu đồ tần suất.
- **Nền ít vân** — tường trắng, sàn phẳng. Không có đặc trưng thì không có từ.

Hai chế độ đầu là chỗ mà mô tả học được thật sự thắng. Chế độ thứ ba thì không: một mô tả toàn ảnh tốt hơn vẫn không phân biệt được hai nơi thật sự giống nhau. Đó là việc của tầng ba và của ràng buộc thời gian.

↔ **tiền đề F/28** — túi từ kế thừa mọi điểm gãy của mô tả cục bộ mà nó gom lại, nên chương 28 giải thích trước nguyên nhân gốc của ba trong bốn chế độ hỏng ở đây

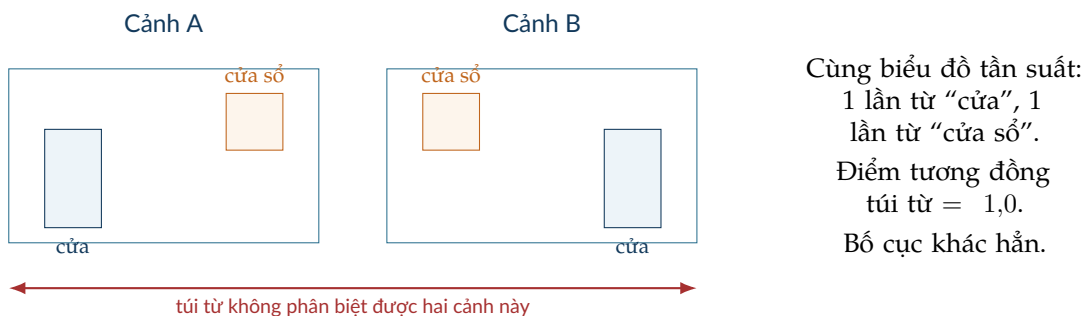
B. Vì sao túi từ mất bố cục, và điều đó có nghĩa gì

Trong bốn chế độ hỏng ở trên, chế độ cấu trúc lặp đáng nhìn kỹ, vì nó không phải lỗi của mô tả ORB mà là lỗi của phép gộp. Túi từ đếm xem mỗi từ thị giác xuất hiện bao nhiêu lần, rồi vứt bỏ hoàn toàn thông tin về vị trí của chúng. Hai bức ảnh chứa cùng các thành phần nhưng sắp xếp khác nhau cho cùng một biểu đồ tần suất.

Hình 1.4 vẽ trường hợp nhỏ nhất. Hai cảnh khác nhau, cùng số cửa và cùng số cửa sổ, chỉ đổi chỗ trái phải. Điểm tương đồng của túi từ giữa chúng bằng đúng một, tức là chúng không phân biệt được.

Điều này giải thích một quan sát thực tế hay gặp: mọi cải tiến ở tầng mô tả cục bộ đều không chứa được perceptual aliasing. Muốn chứa thì phải đưa lại thông tin không gian vào, và có ba cách. Cách rẻ nhất là ràng buộc thời gian ở mục trước. Cách thứ hai là kiểm chứng hình học ở tầng ba, vốn chính là kiểm tra bố cục một cách nghiêm ngặt. Cách thứ ba là xếp hạng lại bằng mảnh đặc trưng có vị trí, tức hướng của Patch-NetVLAD.

Đây cũng là lý do một mô tả toàn ảnh học được không tự nó xoá được perceptual aliasing. Nó gộp tốt hơn, nó bất biến với ánh sáng tốt hơn, nhưng nó vẫn là một vectơ duy nhất cho cả ảnh. Hai nơi thật sự giống nhau vẫn cho hai vectơ gần nhau, và đúng như vậy mới là hành vi đúng của một bộ mô tả.



Hình 1.4. Trường hợp nhỏ nhất cho thấy túi từ mất bố cục không gian. Hai cảnh chứa đúng cùng các thành phần, chỉ đổi vị trí trái phải, nên biểu đồ tần suất từ thị giác của chúng trùng nhau hoàn toàn. Đây là lý do mọi cải tiến ở tầng mô tả cục bộ đều không chứa được perceptual aliasing, và cũng là lý do một mô tả toàn ảnh học được cũng không chứa được.

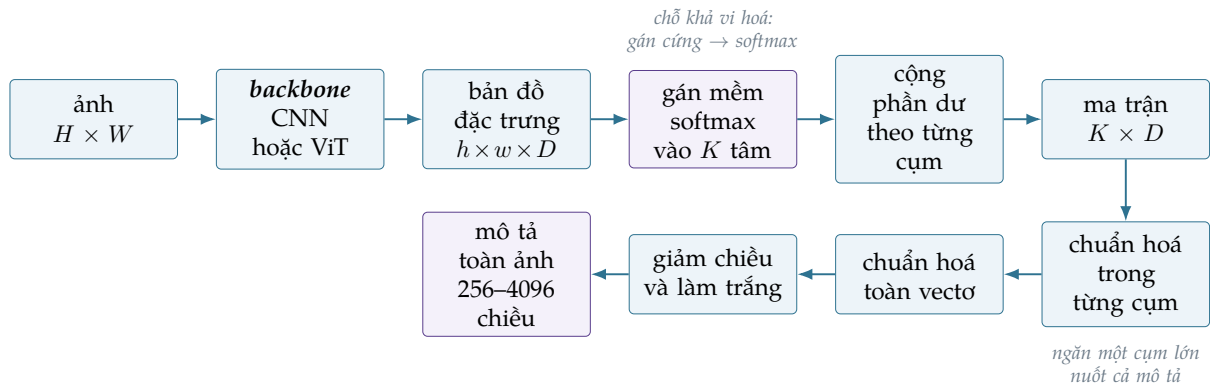
C. NetVLAD: biến phép gộp thành thứ học được

Ý tưởng của VLAD có trước học sâu. Thay vì đếm số đặc trưng rơi vào mỗi cụm, VLAD cộng *phần dư* giữa đặc trưng và tâm cụm. Kết quả là một ma trận $K \times D$, giữ được nhiều thông tin hơn hẳn một biểu đồ tần suất.

Vấn đề là phép gán đặc trưng vào cụm là gán cứng, nên không khả vi, nên không huấn luyện được đầu-cuối. NetVLAD thay gán cứng bằng gán mềm dạng softmax [6]. Chỉ một thay đổi đó làm cả khối gộp trở nên khả vi, và từ đó cả mạng trích đặc trưng lẫn các tâm cụm đều học được cùng lúc.

Điểm thứ hai quan trọng không kém là tín hiệu huấn luyện. NetVLAD học từ ảnh phổ chụp cùng một chỗ ở các thời điểm và góc nhìn khác nhau, với hàm mất mát dạng xếp hạng bộ ba. Nói cách khác, mạng được dạy trực tiếp cái bất biến mà ta cần: cùng nơi thì gần nhau, dù ánh sáng và góc nhìn đổi. Túi từ không có cách nào học điều đó, vì nó không có tham số nào để học.

Hình 1.5 vẽ đường đi của dữ liệu. Chỗ đáng chú ý là hai bước chuẩn hoá cuối: chuẩn hoá trong từng cụm trước, rồi chuẩn hoá toàn vectơ. Bước đầu ngăn một cụm chiếm ưu thế chỉ vì nó có nhiều đặc trưng, tức là ngăn một mảng tường lớn nuốt cả mô tả.



Hình 1.5. Đường đi của dữ liệu trong NetVLAD. Toàn bộ khác biệt so với VLAD cổ điển nằm ở một khối: phép gán đặc trưng vào cụm được làm mềm bằng softmax, nên khả vi, nên cả tâm cụm lẫn mạng trích đặc trưng đều học được. Hai bước chuẩn hoá cuối không phải trang trí; bỏ bước chuẩn hoá trong cụm thì một vùng ảnh đồng nhất lớn sẽ chi phối mô tả.

D. Dòng thời gian: mỗi bước gỡ nút thắt nào

Sau NetVLAD, dòng nghiên cứu này đi theo một mạch rất rõ. Điều đáng chú ý là nút thắt dịch chuyển liên tục, và không phải lúc nào nó cũng nằm ở kiến trúc.

I Mạch 1 – từ túi từ tới đặc trưng nền

- 1 Vấn đề** Túi từ mất hoàn toàn tính bất biến với ánh sáng và góc nhìn lớn, vì nó thừa hưởng mô tả thủ công.

Giải pháp Học phép gộp bằng cách làm nó khả vi, rồi huấn luyện trên ảnh cùng nơi khác thời điểm. Đó là NetVLAD.

Giá phải trả Một lượt xuôi qua mạng cho mỗi khung hình khoá, và một mô tả vài nghìn chiều thay cho vectơ thưa.

Vấn đề mới Mô tả toàn ảnh vẫn không có thông tin bố cục, nên vẫn nhảm ở cấu trúc lặp.
- 2 Vấn đề** Truy hồi toàn ảnh xếp hạng sai khi hai nơi khác nhau có cùng thống kê đặc trưng.

Giải pháp Xếp hạng lại danh sách ngắn bằng chính các mảnh đặc trưng trích từ NetVLAD, có tính tới vị trí không gian. Đó là Patch-NetVLAD [7].

Hệ quả Ranh giới giữa tầng một và tầng ba bắt đầu mờ đi: một phần kiểm chứng bị kéo lên tầng truy hồi.

Giá phải trả Chi phí xếp hạng lại tỉ lệ với số ứng viên, nên ngân sách thời gian thực bị ràng buộc lại.
- 3 Vấn đề** Chạy một mạng cho truy hồi và một mạng khác cho đặc trưng cục bộ là trả hai lần cho cùng một bức ảnh.

Giải pháp Một mạng, một *backbone* chung, ba đầu ra: mô tả toàn ảnh, điểm đặc trưng, mô tả cục bộ. Đó là HF-Net [8].

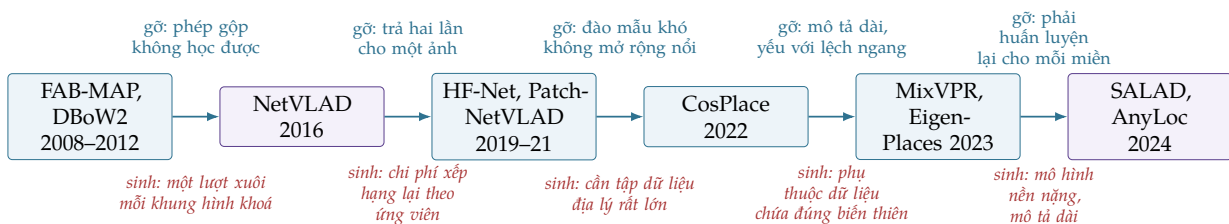
Giả định Ba nhiệm vụ đó dùng chung được biểu diễn ở tầng giữa.

Hệ quả Đây là bài học thiết kế đáng giá nhất của cả mục này với một hệ có ngân sách độ trễ chặt.
- 4 Vấn đề** Huấn luyện bằng xếp hạng bộ ba đòi đào mẫu khó, rất tốn và rất khó ổn định ở quy mô lớn.

Giải pháp Đổi bài toán thành phân lớp: chia thế giới thành các ô theo vị trí và hướng, mỗi ô là một lớp. Đó là CosPlace [9].

Hệ quả	Nút thắt hoá ra nằm ở quy trình dữ liệu và hàm mục tiêu, không nằm ở kiến trúc.
5 Vấn đề	Khối gộp NetVLAD nặng và mô tả đầu ra rất dài.
Giải pháp	Thay gộp theo cụm bằng một chồng khối trộn đặc trưng đơn giản, cho mô tả gọn hơn. Đó là MixVPR [10].
Vấn đề mới	Mô tả gọn hơn nhưng độ bền với lệch góc nhìn ngang chưa được giải quyết riêng.
6 Vấn đề	Lệch góc nhìn ngang, kiểu đi lại cùng con đường ở làn khác, vẫn phá truy hồi.
Giải pháp	Dựng các lớp huấn luyện sao cho mỗi lớp gồm nhiều hướng nhìn khác nhau của cùng một nơi. Đó là EigenPlaces [11].
Giả định	Bất biến góc nhìn học được nếu và chỉ nếu dữ liệu huấn luyện chứa đúng loại biến thiên đó.
7 Vấn đề	Gán mềm của NetVLAD vẫn ép mọi đặc trưng phải thuộc về một cụm nào đó, kể cả đặc trưng vô nghĩa.
Giải pháp	Xem việc gán đặc trưng vào cụm như một bài toán vận tải tối ưu, có thêm một cụm “thùng rác” cho đặc trưng vô ích. Đó là SALAD [12].
Giả định	Đặc trưng đầu vào đã đủ tốt, vì SALAD đặt trên nền DINOv2 [13].
8 Vấn đề	Mọi phương pháp trên đều cần huấn luyện lại khi ra khỏi phân phối: dưới nước, trên không, trong nhà ban đêm.
Giải pháp	Bỏ hẳn việc huấn luyện chuyên biệt. Lấy đặc trưng trung gian của một mô hình nền rồi gộp bằng VLAD cổ điển với từ điển dựng theo miền. Đó là AnyLoc [14].
Giá phải trả	Mô tả dài hơn và một lượt xuôi qua mô hình nền, tức đắt hơn về tính toán.
Hệ quả	Với một robot phải chạy ở miền chưa từng thấy, đây thường là lựa chọn an toàn hơn một mô hình đã tinh chỉnh chặt.

Hình 1.6 tóm mạch trên thành một trục. Điều đáng rút ra không phải thứ tự các tên, mà là việc nút thắt nhảy qua lại giữa ba chỗ: phép gộp, dữ liệu huấn luyện, và đặc trưng đầu vào.



Hình 1.6. Dòng thời gian của mô tả toàn ảnh. Hàng trên ghi nút thắt mà mỗi bước gỡ được; hàng dưới ghi vấn đề mới mà nó tạo ra. Nút thắt nhảy giữa ba chỗ: phép gộp, dữ liệu huấn luyện, và chất lượng đặc trưng đầu vào. Hai ô tô đậm là hai bước đổi hẳn cách nghĩ, không chỉ đổi con số.

E. So sánh có cột “hông khi nào”

Bảng 1.1 đặt các họ cạnh nhau. Cột cuối là cột đáng đọc nhất, vì nó cho biết bạn sẽ mất gì khi đổi.

Bảng 1.1. Các họ mô tả toàn ảnh, so theo cơ chế, giả định và chế độ hỏng. Cột chi phí cố ý ghi định tính: nó nói *loại* chi phí bạn phải trả cho mỗi khung hình khoá, không nói bao nhiêu mili-giây. Con số mili-giây chỉ có nghĩa khi đo trên chính phần cứng đích với hệ đang chạy đủ tải, và đó là việc của bạn chứ không phải của một bảng.

Họ	Cơ chế	Mạnh khi nào	Hỏng khi nào	Chi phí
DBoW2	Biểu đồ tần suất từ thị giác nhị phân, chỉ mục ngược, tf-idf	Cùng điều kiện chụp, cùng hướng đi, bản đồ vừa phải	Đổi ánh sáng mạnh, lệch góc nhìn lớn, cấu trúc lặp, nền ít vân	chi phí biên gần như bằng 0: dùng lại mô tả ORB
NetVLAD	Gộp phần dư quanh K tâm, gán mềm khả vi	Đổi thời điểm và ánh sáng, ngoài trời đô thị	Miền khác hẳn dữ liệu huấn luyện; vẫn nhằm ở cấu trúc lặp	một lượt xuôi mạng cỡ vừa mỗi khung hình khoá
Patch-NetVLAD	Xếp hạng lại danh sách ngắn bằng mảnh đặc trưng có vị trí	Cần precision cao và danh sách ngắn nhỏ	Chi phí tăng tuyến tính theo số ứng viên; không hợp ngân sách chặt	cao, theo ứng viên
HF-Net	Một <i>backbone</i> , ba đầu: toàn ảnh, điểm, mô tả cục bộ	Khi bạn cũng cần đặc trưng cục bộ học được	Ba nhiệm vụ tranh nhau sức chứa nếu <i>backbone</i> quá nhỏ	một lượt chung
CosPlace, MixVPR, EigenPlaces	Đổi hàm mục tiêu và cách dựng lớp huấn luyện	Bản đồ rất lớn; lệch góc nhìn ngang	Vẫn là mô hình huấn luyện theo miền; ra ngoài phân phối thì tụt	cùng bậc với NetVLAD
SALAD	Gán đặc trưng vào cụm bằng vận tải tối ưu, có thùng rác	Nền DINOv2 sẵn; cần mô tả gọn và mạnh	Phụ thuộc chất lượng mô hình nền bên dưới	một lượt xuôi mô hình nền: nặng hơn hẳn
AnyLoc	Đặc trưng mô hình nền, không huấn luyện lại, gộp VLAD	Miền lạ: dưới nước, trên không, trong nhà ban đêm	Khi có nhiều dữ liệu đúng miền, mô hình tinh chỉnh vẫn hơn	như SALAD, cộng mô tả dài hơn

F. Tính tay ba khoản chi phí, và khoản duy nhất đáng kể

Lập luận “túi từ nhẹ hơn” được nhắc rất nhiều nhưng hiếm khi được tính ra số. Có ba khoản chi phí, và chỉ một khoản thật sự quan trọng.

Bộ nhớ mô tả. Đây là một phép nhân chia thuận tuý, và nó cho một kết luận đi ngược trực giác. Giả sử mỗi khung hình khoá giữ khoảng một nghìn đặc trưng — cấu hình mặc định quen thuộc. Vectơ túi từ thừa của nó có tối đa một nghìn cặp gồm mã từ và trọng số; ở tám byte một cặp, đó là cỡ tám kilobyte. Một mô tả toàn ảnh 4096 chiều dạng số thực 32 bit là mười sáu kilobyte, tức lớn hơn. Nhưng nó là vectơ *dày* có độ dài cố định, nên giảm chiều xuống 256 rồi lượng tử hoá về 8 bit là hợp lệ, và khi đó nó chỉ còn vài trăm byte. Vectơ thừa của túi từ không chịu được phép nén tương tự, vì mỗi mục của nó là một cặp chỉ số–trọng số cần thiết cho việc tra chỉ mục ngược. Kết luận về *hình dạng* quan hệ: khi bản đồ lớn dần, mô tả học được là thứ nén được, còn túi từ thì không — và tỉ số nghiêng về phía học được, chứ không phải ngược lại như người ta hay nghĩ.

Chi phí truy vấn. Chỉ mục ngược chỉ duyệt các khung hình khoá có chung ít nhất một từ với truy vấn. Nó rẻ khi từ vựng phân biệt tốt và đắt dần khi bản đồ lớn. Ở phía kia, tìm láng giềng gần nhất là một phép nhân ma trận với vectơ, tức số phép tính tỉ lệ với số khung hình khoá nhân số chiều — một đại lượng nhỏ so với mọi thứ khác trong hệ, và có thư viện vectơ hoá sẵn. Với cả hai bên, nếu bạn nghi ngờ thì hãy đo; nhưng đây hiếm khi là chỗ nút thắt nằm.

Chi phí trích mô tả. Đây là khoản duy nhất đáng kể, và chênh lệch là tuyệt đối chứ không phải theo tỉ lệ. Túi từ trả bằng *không*, vì nó dùng lại đúng các mô tả ORB đã được trích cho việc bám vết. Một mô hình học được trả trọn một lượt xuôi qua mạng cho mỗi khung hình khoá, và lượt xuôi đó là một khoản mới hoàn toàn trong ngân sách. Bao nhiêu mili-giây thì chỉ có phép đo trên phần cứng đích mới nói được, và phải đo khi hệ đang chạy đủ tải chứ không đo mô-đun đứng một mình.

Vậy câu “DBoW2 nhẹ hơn” là đúng, nhưng đúng vì một lý do khác với lý do người ta hay nêu. Nó nhẹ vì chi phí biên của nó bằng không, không phải vì nó tốn ít bộ nhớ. Phân biệt này quan trọng khi bạn cân nhắc: nếu hệ của bạn đã chạy một mạng khác trên khung hình khoá, chi phí biên của việc thêm một đầu ra toàn ảnh có thể gần bằng không, và toàn bộ lập luận đảo chiều. Đó chính là ý tưởng của HF-Net.

G. Thử một bộ truy hồi mà không đụng vào mã của hệ

Có một cách rẻ để trả lời câu “mô tả này có hơn không” trước khi viết một dòng mã tích hợp. Ý tưởng là tách hoàn toàn việc đánh giá tầng một ra khỏi hệ SLAM, và làm nó ngoại tuyến trên chính dữ liệu của bạn.

Quy trình gồm bốn bước.

1. Chạy hệ như bình thường, xuất ra ảnh của mọi khung hình khoá cùng tư thế ước lượng của chúng.
2. Dựng nhãn chân trị cho từng cặp khung hình khoá: đúng nếu khoảng cách thật dưới ngưỡng và hai hướng nhìn không ngược nhau.
3. Với mỗi bộ mô tả cần so, tính vectơ cho mọi khung hình khoá rồi lấy danh sách láng giềng gần nhất.
4. Vẽ recall theo số ứng viên cho từng bộ mô tả, trên cùng một tập khung hình khoá.

Bước bốn cho một đường cong so sánh trực tiếp, trên đúng dữ liệu của bạn, mà không cần tích hợp gì. Nếu hai đường cong gần trùng nhau, câu trả lời đã có và bạn tiết kiệm được vài tuần.

Hai lưu ý làm phép so này trung thực. Thứ nhất, phải loại các cặp khung hình khoá gần nhau về thời gian, nếu không mọi bộ mô tả đều đạt recall gần một và đường cong vô nghĩa. Thứ hai, tập khung hình khoá của bạn được sinh ra bởi chính chính sách chọn khung hình khoá hiện tại, nên nó không phải mẫu ngẫu nhiên của cảnh. Kết luận rút ra chỉ có hiệu lực cho chính chính sách đó.

H. Có đáng thay DBoW2 trên hệ của bạn không

Đây là chỗ phải trả lời thẳng. Câu trả lời phụ thuộc hoàn toàn vào ba con số ở mục trước, chứ không phụ thuộc bảng xếp hạng nào.

Nếu tầng một của bạn đang để lọt phần lớn ứng viên đúng xuống tầng sau, thì thay nó không mua được gì. Đây là điều bảng sống sót trả lời trong một buổi chiều: nếu tỉ lệ sống sót ở cổng một và cổng hai đều cao, tầng một không phải nút thắt, và một bộ truy hồi tốt hơn chỉ đưa thêm ứng viên vào một cái phễu vốn đang nghẽn ở chỗ khác. Trên một hệ chạy chuỗi ngắn cùng điều kiện ánh sáng, đây là kết cục thường gặp — nhưng bạn phải tự đo chứ đừng tin câu này.

Có bốn tình huống mà mô tả học được thật sự thắng, và nên kiểm xem hệ của bạn có rơi vào tình huống nào không.

- **Đi lại sau nhiều giờ hoặc nhiều ngày** — ánh sáng đối hản. Đây là điểm mạnh rõ nhất và cũng là thứ EuRoC không có.
- **Đi lại theo chiều ngược hoặc lệch ngang** — mô tả nhị phân gây sớm hơn nhiều so với mô tả học được.

- **Bản đồ rất lớn hoặc nhiều phiên** — khi số khung hình khoá lên hàng trăm nghìn, phân biệt của tf-idf bão hoà.
- **Định vị lại nguội trong bản đồ đã dựng sẵn** — không có tiên nghiệm chuyển động nào để dựa vào.

Về chi phí, có một tình tiết giảm nhẹ đáng biết: mô tả toàn ảnh chỉ cần tính trên *khung hình khoá*, không phải trên mọi khung hình, nên tần suất gọi thấp hơn tần số ảnh cả một bậc. Ngân sách vì thế thường khả thi, nhưng con số cụ thể phải đo trên phần cứng đích. Cái phải bỏ đi thì rõ ràng hơn và không cần đo: bạn mất khả năng dùng lại mô tả ORB miễn phí, mất tính suy luận được của từ điển, và thêm một phụ thuộc thời gian chạy vào một trình biên dịch suy luận.

CẦN NHỚ

Chỉ thay tầng lấy ứng viên khi bạn đã đo được rằng ứng viên đúng đang chết ở *tầng đó*. Trên một hệ stereo-inertial chạy chuỗi ngắn cùng điều kiện ánh sáng, hãy vào cuộc với giả thuyết ngược lại — rằng tầng đó không phải thủ phạm — rồi bắt bảng sống sót bác bỏ giả thuyết ấy. [T2]

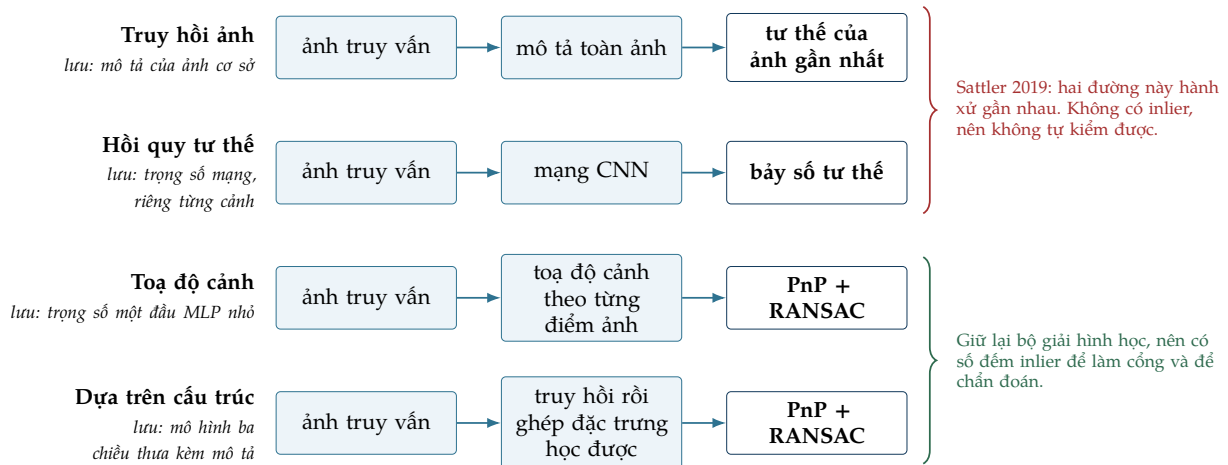
1.4 Định vị lại: hồi quy tư thế, hồi quy tọa độ cảnh và hloc (Relocalization)

A. Bốn họ, phân biệt bằng “cái gì được lưu”

Định vị lại là bài toán anh em của đóng vòng: cho một ảnh và một bản đồ, tìm tư thế camera. Khác biệt là đóng vòng có tiên nghiệm chuyển động, còn định vị lại nguội thì không. Có bốn họ giải bài toán này, và cách phân biệt gọn nhất là hỏi *cái gì được lưu trong bản đồ*.

- **Truy hồi ảnh** — lưu mô tả toàn ảnh của ảnh cơ sở. Trả về tư thế của ảnh gần nhất, hoặc nội suy vài ảnh gần nhất. Không cần hình học.
- **Hồi quy tư thế tuyệt đối** — lưu trọng số một mạng. Mạng nhận ảnh và trả thẳng ra bảy số. Đó là PoseNet [15].
- **Hồi quy tọa độ cảnh** — lưu trọng số một mạng, nhưng mạng trả về tọa độ ba chiều của *từng điểm ảnh* trong hệ tọa độ cảnh. Tư thế được giải sau đó bằng PnP và RANSAC. Đó là dòng DSAC và ACE.
- **Dựa trên cấu trúc** — lưu một mô hình ba chiều thưa kèm mô tả. Ghép hai chiều với ba chiều rồi giải PnP. Đó là hloc và cũng chính là cách ORB-SLAM3 định vị lại.

Hình 1.7 vẽ bốn đường đi này cạnh nhau. Trục ngang là “bao nhiêu hình học còn được giữ ở cuối đường ống”.



Hình 1.7. Bốn họ định vị lại, phân biệt bằng thứ được lưu trong bản đồ và bằng lượng hình học còn lại ở cuối đường ống. Hai họ dưới giữ một bộ giải hình học có kiểm chứng nên có inlier để chẩn đoán; hai họ trên không có. Đó là luận điểm của Sattler và cộng sự ở dạng sơ đồ: hồi quy tư thế hành xử gần với truy hồi ảnh hơn là với định vị dựa trên cấu trúc.

B. PoseNet và phản bác của Sattler 2019

PoseNet đưa ra một ý tưởng rất gọn: huấn luyện một mạng tích chập hồi quy thẳng từ ảnh sang tư thế [15]. Bản đồ trở thành trọng số mạng, kích thước cố định, truy vấn có thời gian cố định. Nghe rất hợp với robot.

Phản bác đến vào năm 2019 và nó rất sắc [16]. Nhóm tác giả dựng một đường cơ sở chỉ dùng truy hồi ảnh cộng nội suy tư thế trên tập huấn luyện. Đường cơ sở đơn giản đó ngang bằng hoặc tốt hơn phần lớn phương pháp hồi quy tư thế. Họ còn chỉ ra rằng sai số của hồi quy tư thế tăng vọt khi tư thế truy vấn ra ngoài phân phối tư thế huấn luyện.

Cơ chế đằng sau rất dễ hiểu khi đã nói ra. Mạng không học hình học chiều; nó học một ánh xạ từ ảnh vào đa tạp các tư thế đã thấy khi huấn luyện. Vì vậy nó nội suy tốt và ngoại suy rất tệ. Về mặt hành vi, nó là một bộ truy hồi có nội suy, chứ không phải một bộ định vị.

Với kỹ sư SLAM, hệ quả thực dụng còn quan trọng hơn kết quả số. Hồi quy tư thế không trả về inlier. Không có inlier thì không có cách nào biết một tư thế trả về là đáng tin hay rác. Trong một hệ mà một ràng buộc sai làm hỏng cả bản đồ, mất khả năng tự kiểm ấy là cái giá quá đắt.

↔ cùng nguyên lý với B/08 — đây đúng là dịch chuyển phân phối ở dạng hình học: tư thế truy vấn ngoài bao lồi của tư thế huấn luyện là một mẫu ngoài phân phối

C. Định vị lại nguội trong hệ của bạn, và ba chỗ chen được mô-đun học được

Mọi so sánh phải so với đường mà hệ của bạn đang chạy, nên cần nhắc lại đường đó trước. ORB-SLAM3 khi mất bám vết sẽ truy vấn túi từ để lấy ứng viên, ghép đặc trưng theo cây từ vựng, chạy PnP trong RANSAC để có tư thế thô, rồi tìm thêm khớp bằng cách chiếu bản đồ và tối ưu lại [1]. Đó vẫn là bốn tầng cũ, chỉ khác ở chỗ bài toán là sáu bậc chứ không phải bảy.

Có ba chỗ chen được một mô-đun học được vào đường đó, và rủi ro của chúng rất khác nhau.

- **Chỉ thay tầng một** — thêm một mô tả toàn ảnh học được, chạy song song với túi từ, rồi hợp nhất hai danh sách ứng viên. Rủi ro thấp nhất, vì tầng ba vẫn kiểm chứng mọi thứ. Nếu mô-đun mới đề xuất rác, rác bị loại.
- **Thay tầng hai cho riêng cặp ứng viên** — chạy bộ ghép cặp học được thay cho ghép theo cây từ vựng. Rủi ro trung bình, vì nó đổi phân bố inlier nên buộc phải đặt lại ngưỡng ở tầng ba.

- **Thay cả tầng ba** — dùng một mô-đun trả thẳng tư thế. Rủi ro cao nhất, vì bạn mất số đếm inlier, tức mất luôn cái cổng.

Thứ tự rủi ro này cũng là thứ tự nên thử. Nó cho phép bạn dừng ngay khi đo được rằng bước tiếp theo không mua thêm gì.

D. Từ DSAC tới ACE: giữ lại bộ giải hình học

Hồi quy toạ độ cảnh chọn một chỗ đứng khác. Mạng không dự đoán tư thế; nó dự đoán, cho mỗi điểm ảnh, toạ độ ba chiều tương ứng trong hệ toạ độ cảnh. Tư thế được giải sau bằng PnP trong vòng RANSAC. Bộ giải hình học được giữ nguyên.

DSAC giải quyết trở ngại kỹ thuật của cách làm này: RANSAC không khả vi, nên không lan được *gradient* về mạng [17]. Giải pháp là thay bước chọn giả thuyết tốt nhất bằng một phép lấy mẫu theo xác suất, rồi lấy kỳ vọng. Nhờ vậy toàn bộ đường ống huấn luyện được đầu-cuối.

DSAC++ bỏ được yêu cầu phải có mô hình ba chiều khi huấn luyện, bằng cách khởi tạo qua sai số chiếu lại [18]. DSAC* hợp nhất các biến thể ảnh màu và ảnh có độ sâu, đồng thời rút ngắn huấn luyện [19].

ACE là bước có ý nghĩa thực dụng nhất của dòng này [20]. Nó tách mạng thành hai phần: một *backbone* trích đặc trưng, huấn luyện sẵn và không phụ thuộc cảnh, cộng một đầu MLP nhỏ riêng cho từng cảnh. Chỉ đầu MLP được huấn luyện lại, trên các *batch* rất lớn gồm đặc trưng lấy ngẫu nhiên khắp tập ảnh. Kết quả là bản đồ của một cảnh thu về đúng trọng số của một đầu MLP nhỏ, và thời gian dựng nó ngắn hơn hẳn một quy trình dựng lại cấu trúc từ chuyển động.

ACE Zero đi thêm một bước: học toạ độ cảnh từ tập ảnh *không có tư thế*, tức là thay luôn vai trò của một quy trình dựng lại cấu trúc từ chuyển động [21]. Dòng này còn tiếp tục với các biến thể mang tên ACE-G và ACE-SLAM, hướng tới tổng quát hoá sang cảnh mới và tới việc nhúng thẳng vào vòng SLAM. Tôi chưa đối chiếu chi tiết hai công trình đó, nên chỉ nêu tên và ý tưởng, không trích dẫn và không mô tả kết quả số của chúng.

E. hloc: chính cái phễu của bạn, thay từng tầng bằng mô-đun học được

hloc là quy trình định vị đang được dùng như chuẩn tham chiếu [8]. Nó gồm bốn bước: truy hồi bằng mô tả toàn ảnh, trích đặc trưng cục bộ học được, ghép cặp học được, rồi PnP trong RANSAC trên một mô hình ba chiều dựng bằng COLMAP [22].

Điều cần thấy là hloc *chính là* cái phễu bốn tầng ở Hình 1.1, với ba tầng đầu được thay bằng mô-đun học được và tầng cuối giữ nguyên bộ giải hình học. Đây là kiến trúc an toàn, và nó là bài học thiết kế lớn nhất của cả chương: học phần tìm tương ứng, giữ nguyên phần ước lượng.

Lý do rất cụ thể. Bộ ước lượng hình học cho bạn ba thứ mà một mạng hồi quy không cho: một số đếm inlier để làm cổng, một hiệp phương sai để đặt trọng số trong đồ thị nhân tử, và khả năng chẩn đoán khi hỏng. Bỏ cả ba thứ đó để đổi lấy vài phần trăm recall là một cuộc đổi chác tồi với hệ đang chạy thật.

Về chi phí, hloc không được thiết kế cho thời gian thực và cũng không nên bị ép vào đó. Điều cần thấy là *cấu trúc* của chi phí chứ không phải một con số: một truy vấn đầy đủ gồm một lượt trích mô tả toàn ảnh, một lượt trích đặc trưng cục bộ, rồi *một lượt ghép cặp cho mỗi ảnh trong danh sách ngắn*. Khoản cuối là khoản nhân lên: danh sách ngắn mười ảnh nghĩa là mười lần chạy bộ ghép cặp cho một truy vấn. Vì thế hai núm vặn duy nhất có ý nghĩa là độ dài danh sách ngắn và tần suất gọi. Cách dùng thực tế trong một hệ SLAM là cắt danh sách ngắn xuống hai hoặc ba, và chỉ chạy khi đã mất tám vết hoặc khi có một ứng viên vòng, chứ không chạy mỗi khung hình.

F. Gộp bản đồ: cùng chuỗi cổng, người tiêu thụ khác

Có một chi tiết kỹ thuật quyết định việc mô-đun định vị lại nào cắm được vào hệ của bạn. ORB-SLAM3 dùng đúng một chuỗi cổng cho hai việc khác nhau: đóng vòng trong cùng một bản đồ, và gộp hai bản đồ rời [1]. Khác biệt nằm ở đầu ra chứ không ở đầu vào.

Khi gộp hai bản đồ, hai bản đồ có hai gốc tọa độ độc lập và, với hệ đơn ảnh, hai tỉ lệ độc lập. Vì vậy phép biến đổi cần tìm bắt buộc là Sim3 bảy bậc, không phải một tư thế tuyệt đối. Với hệ stereo hoặc stereo-inertial, tỉ lệ trong mỗi bản đồ đã quan sát được nên hệ số tỉ lệ gần một, nhưng nó vẫn phải được ước lượng chứ không được giả định.

Hệ quả cho các phương pháp học được rất thẳng. Một mô-đun chỉ trả về tư thế tuyệt đối trong một hệ tọa độ cố định thì không dùng được cho việc gộp bản đồ, vì nó không nói gì về quan hệ giữa hai bản đồ. Đây là một lý do kỹ thuật, không phải lý do độ chính xác, và nó thường bị bỏ qua khi người ta so bảng số của các phương pháp định vị lại.

G. Cổng đánh giá cho hệ của bạn

Bảng 1.2 tóm điều kiện dùng được. Kết luận ngắn: hồi quy tọa độ cảnh có chỗ đứng thật, nhưng không phải trong vòng SLAM trực tuyến.

Bảng 1.2. Bốn họ định vị lại, đọc từ góc nhìn một hệ SLAM đang chạy. Cột cuối nói thẳng khi nào không nên dùng.

Họ	Bản đồ là gì	Mạnh khi nào	Hỏng khi nào
Truy hồi ảnh	Mô tả toàn ảnh của ảnh cơ sở	Cần ứng viên nhanh; chấp nhận sai số tư thế lớn	Không cho tư thế chính xác, không có inlier để kiểm chứng
Hồi quy tư thế	Trọng số mạng, riêng từng cảnh	Cảnh nhỏ, tư thế truy vấn nằm trong phân phối huấn luyện	Ngoại suy ngoài quỹ đạo huấn luyện; không có cách tự kiểm
Tọa độ cảnh	Trọng số một đầu MLP nhỏ	Toà nhà đã biết, dựng bản đồ một lần, cần bản đồ rất gọn	Bản đồ đang mở rộng trực tuyến; cần Sim3 để gộp hai bản đồ
Dựa trên cấu trúc	Mô hình ba chiều thừa kèm mô tả	Cần tư thế chính xác kèm inlier; cần gộp bản đồ	Bản đồ rất lớn thì tốn bộ nhớ; ghép cặp học được thì tốn tính toán

Ba lý do khiến hồi quy tọa độ cảnh chưa vào được vòng SLAM trực tuyến. Thứ nhất, nó cần huấn luyện lại cho mỗi cảnh, trong khi bản đồ SLAM lớn dần theo thời gian. Thứ hai, nó trả về tư thế trong một hệ tọa độ cố định, còn đóng vòng cần một phép Sim3 giữa hai đoạn bản đồ. Thứ ba, nó giả định cảnh tĩnh và đã được quan sát đủ khi huấn luyện.

Ngược lại, có một tình huống triển khai mà nó rất hợp: một toà nhà cố định, quét một lần, rồi robot chạy trong đó nhiều tháng. Khi đó việc huấn luyện một đầu MLP cho mỗi khu là chi phí trả một lần, và một bản đồ gọn hơn hẳn mô hình ba chiều thừa là lợi thế thật.

1.5 Kiểm chứng: hình học so với học được (Verification)

A. Vì sao RANSAC Sim3 là chỗ hẹp nhất

Ở tầng ba, hệ phải tìm phép biến đổi bảy bậc tự do nối hai đoạn bản đồ. Bảy bậc vì tỉ lệ của bản đồ đơn ảnh trôi theo thời gian, nên hai đầu vòng nói chung khác tỉ lệ [23]. Nghiệm đóng cho ba cặp điểm đã có từ lâu [4], [5], nên tập mẫu tối thiểu là ba.

Ba yếu tố cùng đẩy tăng này vào thế khó.

- **Số cặp đầu vào ít** — tăng hai chỉ đề xuất vài chục cặp, vì việc ghép bị giới hạn trong cùng nút từ vừng. Đây là mẫu số của w , và bạn phải ghi nó lại chứ không được đoán.
- **Tỉ lệ inlier thấp** — lệch góc nhìn lớn làm phần lớn cặp sai. Ngưỡng inlier chia cho mẫu số ấy cho ra w^* , tỉ lệ inlier tối thiểu bắt buộc phải có; với vài chục cặp và ngưỡng vài chục phần trăm của chúng, w^* rơi vào vùng mà đường cong ở Hình 1.3 đang rất dốc.
- **Bậc ba trong công thức** — xác suất một mẫu sạch là w^3 , nên tụt rất nhanh khi w giảm.

Nhìn ba yếu tố này, hướng chữa hiển nhiên là giảm m . Với camera stereo, tỉ lệ đã quan sát được, nên về lý thuyết bài toán rút xuống sáu bậc. Nếu thêm phương trọng lực từ IMU, hai bậc xoay nữa bị ghim, chỉ còn góc phương vị và tịnh tiến, và tập mẫu tối thiểu rút xuống hai. Hình 1.3 cho thấy lời hứa: ở vùng w thấp, khoảng cách giữa hai đường cong rất lớn.

B. Ví dụ tính tay: vì sao ba cặp điểm là số nhỏ nhất

Lấy ba cặp điểm cụ thể. Trong bản đồ thứ nhất, ba điểm là $p_1 = (0, 0, 0)$, $p_2 = (1, 0, 0)$, $p_3 = (0, 1, 0)$. Trong bản đồ thứ hai, chúng lần lượt là $q_1 = (1, 1, 0)$, $q_2 = (1, 3, 0)$, $q_3 = (-1, 1, 0)$.

Bước một, tìm tỉ lệ. Khoảng cách $|p_2 - p_1| = 1$ còn $|q_2 - q_1| = 2$, nên $s = 2$. Kiểm chéo bằng cặp còn lại: $|p_3 - p_1| = 1$ và $|q_3 - q_1| = 2$, cũng cho $s = 2$.

Bước hai, tìm phép xoay. Sau khi chia cho tỉ lệ, vectơ $(1, 0, 0)$ đi thành $(0, 1, 0)$, và vectơ $(0, 1, 0)$ đi thành $(-1, 0, 0)$. Đó là phép xoay 90 độ quanh trục z .

Bước ba, tìm tịnh tiến. Tâm của hai bộ điểm là $\bar{p} = (\frac{1}{3}, \frac{1}{3}, 0)$ và $\bar{q} = (\frac{1}{3}, \frac{5}{3}, 0)$. Khi đó $t = \bar{q} - sR\bar{p} = (1, 1, 0)$. Kiểm lại với điểm thứ hai: $sRp_2 + t = 2(0, 1, 0) + (1, 1, 0) = (1, 3, 0) = q_2$. Đúng.

Bây giờ tới phần đáng giá. Vì sao tập mẫu tối thiểu là ba chứ không phải hai? Mỗi cặp điểm cho ba ràng buộc vô hướng. Sim3 có bảy bậc tự do. Hai cặp cho sáu ràng buộc, thiếu một. Ba cặp cho chín, dư hai. Vì vậy ba là số nhỏ nhất, và không thể nhỏ hơn nếu không có thông tin ngoài.

Câu cuối chính là chỗ ý tưởng dùng trọng lực xuất hiện. Phương trọng lực từ IMU ghim hai bậc xoay, và camera stereo ghim bậc tỉ lệ. Còn lại bốn bậc, cần sáu ràng buộc, nên hai cặp điểm là đủ. Về mặt tổ hợp, đó là một cải thiện rất lớn: xác suất rút trúng đi từ w^3 lên w^2 .

Bài học tổng quát nằm ở đây, và nó vượt ra ngoài đóng vòng. Mỗi thông tin ngoài mà bạn đưa vào một bộ ước lượng đều rút bậc tự do, và mỗi bậc rút được đều làm xác suất rút mẫu sạch tăng theo hàm mũ. Đó là lý do lập luận này nghe rất thuyết phục. Mục sau kể chuyện gì xảy ra khi đem nó đi đo.

C. Ba giả định ẩn của lời hứa đó, và cách đem nó đi đo

Lập luận ở mục trước rất chặt về mặt toán, nên nó dễ khiến người ta bỏ vài tuần cài đặt trước khi kiểm bất cứ điều gì. Đừng làm thế. Công thức $1 - (1 - w^m)^N$ đứng trên ba giả định ẩn, và mỗi giả định vỡ theo một kiểu riêng. Ba giả định ấy đều kiểm được *trước* khi viết dòng mã nào, bằng đúng cái nhậ ký ở Bảng 1.4.

I. Mục 2 — ba giả định ẩn của việc rút tập mẫu tối thiểu

1 Vấn đề

Giả định

Hệ quả

Giả định thứ nhất: inlier có đủ, chỉ thiếu lượt rút trúng.

Toàn bộ công thức nói về xác suất *rút*. Nó im lặng hoàn toàn về việc có tồn tại đủ cặp đúng hay không.

Nếu biểu đồ ở Hình 1.2 mang Dạng B, tức đội trần dưới ngưỡng, thì giả định này sai và hạ m không mua được gì. Phép kiểm: so số inlier tốt nhất của các ứng

viên đúng với w^* nhân số cặp đề xuất. Nếu phần lớn nằm dưới, dừng lại ở đây — nút thất nằm ở tầng hai.

2 Vấn đề

Giả định

Hệ quả

Giả định thứ hai: hai bậc xoay bị ghim bằng trọng lực là *ghim đúng*.

Phương trọng lực ước lượng được từ IMU có sai số, và sai số nghiêng còn dư tại điểm gặp lại không nhất thiết nhỏ.

Một giả thuyết chỉ có góc phương vị sẽ ép phép biến đổi về thẳng đứng. Nếu phần nghiêng còn dư đáng kể, ràng buộc sinh ra sai một cách có hệ thống, và tầng bốn sẽ trung thành bơm cái sai đó ra khắp bản đồ. Phép kiểm: với mỗi vòng đóng được bằng nghiệm bảy bậc, ghi lại phần nghiêng của phép Sim3 tìm được; nếu phân bố của nó không tập trung quanh không, việc ghim hai bậc là sai mô hình.

3 Vấn đề

Giả định

Hệ quả

Giả định thứ ba: thêm vòng thì bản đồ chỉ tốt lên.

Mỗi vòng được chấp nhận kéo theo một lần hiệu chỉnh bản đồ, và mỗi lần hiệu chỉnh là một cửa sổ mà luồng dựng bản đồ cục bộ bị chặn.

Nới một cổng để nhận thêm vòng vì thế đổi cả hai đầu: recall tăng, nhưng số cửa sổ rủi ro cũng tăng. Phép kiểm: đếm số lần mất bám vết và số lần hệ sập, không chỉ đếm số vòng.

Cách đo cả ba giả định trong một thí nghiệm duy nhất thì rẻ, và nó không đòi bạn cài đặt bộ giải hai điểm. Chạy hệ như bình thường với bộ giải ba điểm, nhưng *tăng mạnh số vòng lặp RANSAC* — chẳng hạn nhân lên mười lần — rồi so số vòng đóng được. Nếu nguyên nhân thật là thiếu mẫu thì tăng N đã phải mua được một phần lợi ích, vì N và m đi vào cùng một công thức; nếu tăng N không đổi gì thì hạ m cũng sẽ không đổi gì, và bạn vừa tiết kiệm được vài tuần bằng một dòng cấu hình. Đây là thí nghiệm phải chạy *trước*, và nó đúng tinh thần của thí nghiệm oracle ở E/24: đo trần đóng góp của một hướng trước khi đầu tư vào nó.

Cần ghi kèm điều kiện hiệu lực cho toàn bộ lập luận này. Nó được viết cho một hệ stereo-inertial chạy chuỗi ngắn trong nhà, nơi tỉ lệ trôi tỉ lệ nhỏ và tỉ lệ inlier không quá thấp. Trên hệ đơn ảnh ngoài trời, tỉ lệ inlier thấp hơn nhiều, giả định thứ nhất có nhiều khả năng đúng hơn, và cân nhắc có thể đảo chiều.

D. Cổng thứ năm: cái cổng đứng ngay sau RANSAC, và vì sao nó bị bỏ quên

Cổng RANSAC chiếm hết sự chú ý vì nó là cổng lọc mạnh nhất. Nhưng cổng đứng ngay sau nó — tối ưu Sim3 rồi đếm lại inlier — thường cũng loại một phần đáng kể những gì đã vượt được RANSAC, và gần như không ai đo nó. Đây là chỗ dễ có bất ngờ nhất trong cả chuỗi, đúng vì không ai nhìn.

Có ba nguyên nhân khả dĩ, và chúng đòi ba cách xử lý khác nhau.

- **Nghiem thô quá thô** — Sim3 dựng từ ba cặp điểm nằm sát ngưỡng chỉ chính xác cục bộ. Khi dùng nó để chiếu lại và tìm thêm khớp với bán kính hẹp, phép tìm hụt.
- **Tối ưu rơi vào cực tiểu gần đó** — với ít inlier, hàm mục tiêu nông và có nhiều cực tiểu địa phương gần nhau.
- **Giả thuyết vốn dĩ sai** — một ứng viên sai tình cờ vượt được ngưỡng inlier vẫn có thể bị loại đúng ở đây. Trong trường hợp này, cổng năm đang làm hàng phòng thủ thứ hai.

Tỉ lệ giữa ba nguyên nhân này là một câu hỏi thực nghiệm, và cách trả lời nó giống hệt cách đã dùng cho cổng ba: gắn nhãn chân trị cho các ứng viên chết ở cổng năm rồi tách bảng làm hai. Nếu phần lớn là ứng viên sai, cổng năm là bạn và không nên dựng vào. Nếu phần lớn là ứng viên

đúng, thì hai nguyên nhân đầu là chỗ đáng đầu tư, và cách chữa rẻ nhất là nói bán kính chiếu lại rồi tối ưu lại một lần nữa.

Nguyên tắc chung rút ra được: mỗi khi một cổng loại nhiều, câu hỏi tiếp theo luôn giống nhau, và nó là câu hỏi về nhân chứ không phải về ngưỡng.

E. Năm hướng chữa, xếp theo chỗ chúng tác động

Bảng 1.3 liệt kê các hướng đã biết. Hai hướng đầu không dùng học sâu chút nào.

Bảng 1.3. Năm hướng chữa cổng kiểm chứng, xếp theo tầng mà chúng thật sự tác động. Cột cuối không ghi kết quả của ai cả — nó ghi *điều kiện* để hướng đó có lãi, và phép đo bạn phải chạy để biết điều kiện ấy có thoả trên hệ mình hay không.

Hướng	Tác động ở tầng	Cơ chế	Bằng chứng
Rút mẫu tốt hơn: PROSAC, LO-RANSAC, MAGSAC++	Tầng ba, phần rút mẫu	Xếp thứ tự mẫu theo điểm khớp; tối ưu cục bộ trên tập inlier; bỏ ngưỡng cứng [24]–[26]	Rẻ, không cần huấn luyện. Chỉ giúp nếu biểu đồ inlier mang Dạng A
Giảm bậc tự do bằng trọng lực và stereo	Tầng ba, phần rút mẫu	Ghim tỉ lệ bằng stereo, ghim nghiêng bằng IMU, còn hai cặp điểm	Cùng điều kiện với dòng trên, cộng hai giả định nữa. Kiểm bằng phép thử tăng N ở Mạch 2 trước khi cài
Nói cổng đếm, để tầng sau quyết	Tầng ba, phần ngưỡng	Cho giả thuyết tốt nhất đi tiếp dù dưới ngưỡng, dùng chuỗi cổng sẵn có để lọc	Trần lợi ích bằng đúng số ứng viên đúng nằm trong khoảng trống giữa hai phân bố. Đếm nó trước, rồi mới quyết
Ghép cặp học được cho riêng cặp ứng viên	Tầng hai	SuperGlue hoặc LightGlue trên cặp ảnh, tăng cả số cặp lẫn tỉ lệ inlier [27], [28]	Hướng duy nhất tấn công thẳng trần inlier. Chi phí trả một lần cho mỗi ứng viên
Ghép cặp dày không cần điểm đặc trưng	Tầng hai	LoFTR hoặc RoMa cho tương ứng dày ở đường cơ sở rộng [29], [30]	Mạnh nhất khi lệch góc nhìn lớn; cũng đắt nhất, và chi phí trả theo từng cặp ứng viên

Còn một hướng thứ sáu, rẻ và không ai nói tới, nằm ngay trong tầng hai. Việc ghép cặp hiện bị giới hạn trong cùng nút từ vừng ở một mức cây cố định. Nói mức cây đó, hoặc ghép vết cận có kiểm tra tỉ số khoảng cách chỉ cho riêng cặp ứng viên, sẽ tăng số cặp đề xuất. Chi phí trả đúng một lần cho mỗi ứng viên, không phải cho mỗi khung hình. Đây là suy luận từ cơ chế trần inlier, chưa phải một kết quả đã kiểm chứng — nhưng nó rẻ tới mức đáng thử trước mọi hướng học sâu trong bảng.

F. Kiểm chứng học được: chấm điểm một cặp thay vì giải hình học

Có một hướng khác hẳn ở tầng ba: thay vì giải một phép biến đổi, huấn luyện một mạng chấm điểm “hai ảnh này có phải cùng một nơi không”. Hướng này xuất hiện dưới nhiều tên, thường là xếp hạng lại danh sách ngắn. Patch-NetVLAD thuộc nhóm này, và mọi bộ ghép cặp học được cũng có thể dùng theo kiểu này bằng cách lấy số cặp tin cậy làm điểm.

Điểm mạnh thì rõ. Một mạng có thể học các dấu hiệu mà hình học không dùng được: ngữ cảnh toàn ảnh, kết cấu bề mặt, bố cục thô của cảnh. Nó cũng không bị chặn bởi trần inlier, vì nó không cần một tập cặp nhất quán để đưa ra điểm số.

Nhưng có một khác biệt mà không lập luận nào xoá được. Bộ giải hình học trả về một *con số kiểm chứng được*: số inlier, tức số phép đo độc lập cùng ủng hộ một giả thuyết. Một mạng trả về một điểm số đã hiệu chỉnh kém và không có ý nghĩa vật lý. Với một hệ mà một vòng sai bẽ cả bản đồ, khác biệt đó quyết định.

Vì vậy cách dùng đúng là dùng nó làm *bộ lọc trước*, không phải làm bộ thay thế. Cụ thể: cho mạng chấm điểm và sắp xếp lại danh sách ứng viên, rồi chỉ chạy hình học cho vài ứng viên đầu. Bạn tiết kiệm được phần lớn chi phí ở tầng ba mà không mất cái cổng.

Có một trường hợp mà kiểm chứng học được thật sự cần thiết: khi tỉ lệ inlier thấp tới mức hình học không bao giờ hội tụ, chẳng hạn đi lại sau sáu tháng ở cảnh ngoài trời đổi mùa. Ở đó hình học không có gì để làm việc, nên một điểm số học được vẫn hơn không có gì. Đây không phải chế độ vận hành của một hệ chạy chuỗi ngắn trong nhà.

G. Đưa ghép cặp học được vào tầng kiểm chứng thì đổi gì

SuperGlue coi ghép cặp là bài toán gán tối ưu giữa hai tập điểm, giải bằng một mạng đồ thị có *attention* chéo giữa hai ảnh [27]. LightGlue giữ cùng ý tưởng nhưng cho phép dùng sớm theo độ khó của cặp, nên chi phí thích nghi [28].

Ba thay đổi xảy ra khi đưa chúng vào tầng hai của vòng lặp.

- **Số cặp tăng và tỉ lệ inlier tăng** — đây là tác động mong muốn, và nó đánh đúng vào trần inlier.
- **Chi phí trả theo ứng viên, không theo khung hình** — khác hẳn khi dùng chúng cho bám vết. Đây là tình tiết giảm nhẹ quan trọng nhất: số lần gọi bằng số ứng viên mỗi khung hình khoá, tức nhỏ hơn tần số ảnh nhiều bậc. Con số mili-giây thì phải đo trên phần cứng đích, nhưng *cấu trúc* chi phí đã đủ để nói rằng hướng này khả thi hơn hẳn việc chạy cùng mô hình đó cho bám vết.
- **Điểm vận hành dịch chuyển** — nhiều cặp hơn cũng nghĩa là ứng viên sai có nhiều cơ hội hơn để tạo ra một Sim3 trông hợp lý. Precision có thể tụt.

Điểm thứ ba là chỗ hay bị quên. Khoảng trống giữa hai phân bố trong Hình 1.2 là một tính chất của *bộ ghép cặp hiện tại*, không phải một hằng số của bài toán. Đối bộ ghép cặp là đổi cả mẫu số lẫn phân bố, nên phải đo lại khoảng trống đó và đặt lại ngưỡng theo phân bố mới. Nếu không, bạn nhận thêm recall kèm một tỉ lệ vòng sai chưa được đo — và mục sau giải thích vì sao đó là cuộc đổi chác tồi nhất có thể.

1.6 Đo cho đúng: recall, precision và cái giá của một vòng sai (Measuring Loop Closure)

A. Định nghĩa trước, con số sau

Không có con số recall nào có nghĩa nếu chưa nói ba thứ. Thứ nhất, ngưỡng khoảng cách để gọi hai khung hình là “cùng nơi”. Ngoài trời người ta hay dùng 25 mét; trong nhà con số đó vô nghĩa, phải dùng vài mét. Thứ hai, mẫu số: đếm trên tổng số truy vấn, hay chỉ trên các truy vấn thật sự có lần đi lại. Thứ ba, đơn vị: một khung hình, hay một lần đi lại gồm nhiều khung hình.

Ba định nghĩa này có thể làm cùng một hệ hiện ra với recall rất khác nhau. Vì vậy khi đọc một bảng của người khác, hãy tìm ba định nghĩa đó trước khi đọc con số [31].

Một ví dụ tính tay cho thấy độ nhạy. Giả sử một hệ đóng đúng 40 vòng. Với ngưỡng bốn mét, có 100 lần đi lại được coi là “có thật”, nên recall là 0,40. Nới ngưỡng lên mười mét, số lần đi lại “có thật” tăng lên 150, trong khi số vòng đóng được chỉ tăng lên 45 vì các cặp mới thêm đều lệch

góc nhìn lớn. Recall khi đó là 0,30. Cùng một hệ, cùng một lần chạy, hai con số khác nhau một phần tư chỉ vì đổi định nghĩa. Đây cũng là lý do không so được recall giữa hai bài báo dùng hai ngưỡng khác nhau.

B. Đường cong theo số ứng viên, và giới hạn của nó

Chỉ số chuẩn của tầng một là recall theo số ứng viên: xác suất có ít nhất một ứng viên đúng trong N ứng viên đầu. Đường cong này tăng đơn điệu theo N , nên nó luôn đẹp lên khi ta lấy nhiều ứng viên hơn.

Đó cũng chính là giới hạn của nó. Nó là chỉ số của *một tầng*, không phải của hệ thống. Tầng N làm recall tầng một tăng, đồng thời làm khối lượng việc của tầng ba tăng tuyến tính và làm số ứng viên sai lọt vào tầng ba tăng theo. Một hệ có recall tầng một tuyệt vời vẫn có thể đóng số vòng bằng không.

Vì vậy phải báo cáo hai đường cong cạnh nhau: recall theo số ứng viên ở tầng một, và precision-recall của *toàn bộ chuỗi cổng*. Cặp đường cong này là thứ duy nhất trả lời được câu hỏi “nới ngưỡng thì được gì và mất gì”.

C. Dựng đường precision-recall cho cả chuỗi cổng

Một chuỗi sáu cổng không có một ngưỡng duy nhất để quét, nên việc dựng đường cong cần kỹ luật. Quy trình đúng gồm ba bước.

1. Chọn *một* ngưỡng làm trục quét, thường là ngưỡng inlier ở cổng ba, rồi giữ mọi ngưỡng khác cố định.
2. Với mỗi giá trị, chạy lại chuỗi cổng *trên nhật ký đã ghi*, không chạy lại hệ SLAM.
3. Với mỗi giá trị, đếm vòng được chấp nhận, tra nhãn chân trị, và ghi ra một cặp precision-recall.

Bước hai là bước hay bị làm sai nhất, và làm sai nó khiến cả đường cong vô nghĩa. Nếu bạn chạy lại toàn bộ hệ với ngưỡng mới, quỹ đạo sẽ khác, nên tập khung hình khoá khác, nên tập ứng viên khác. Hai điểm trên đường cong khi ấy không còn so được với nhau, vì chúng đến từ hai tập dữ liệu khác nhau. Đây đúng là quy tắc mỗi lần chỉ đổi một thứ, áp vào một chuỗi cổng.

Muốn quét lại trên nhật ký thì nhật ký phải đủ giàu. Bảng 1.4 liệt kê các trường tối thiểu và nói mỗi trường chẩn đoán được gì. Sáu trường này đủ để tái dựng mọi quyết định của chuỗi cổng một cách ngoại tuyến.

Bảng 1.4. Các trường tối thiểu cần ghi cho mỗi ứng viên. Cột cuối là lý do ghi: mỗi trường trả lời một câu hỏi chẩn đoán mà các trường khác không trả lời được.

Trường	Nội dung	Chẩn đoán được gì
Mã hai khung hình khoá	Định danh truy vấn và ứng viên	Tra chân trị để gắn nhãn đúng-sai
Điểm truy hồi	Điểm số của tầng một	Dựng đường recall theo số ứng viên
Số khớp túi từ	Số cặp mà tầng hai đề xuất	Mẫu số của tỉ lệ inlier; phát hiện trần inlier
Số inlier tốt nhất của RANSAC	Giả thuyết tốt nhất sau 300 vòng lặp	Phân biệt thiếu mẫu với trần inlier
Số khớp sau khi chiếu lại	Kết quả tìm thêm khớp có dẫn hướng	Cho biết nghiệm thô có đủ chính xác không
Số inlier sau tối ưu Sim3	Kết quả cổng năm	Tách ba nguyên nhân chết ở cổng năm

Chỉ số nên báo cáo là recall tại mức precision 100 %. Nó thô, nhưng nó đúng bản chất bài toán: nếu tôi không được phép sai lần nào thì tôi bắt được bao nhiêu phần số vòng có thật. Kèm theo nó phải là một cảnh báo về cỡ mẫu. Với vài chục vòng đúng trong toàn bộ thí nghiệm, đúng một vòng sai đã kéo precision từ 100 % xuống khoảng 97 %. Vì vậy luôn báo cáo số vòng tuyệt đối bên cạnh tỉ lệ phần trăm.

D. Bất đối xứng chi phí: vì sao điểm vận hành lệch hẳn về precision

Đây là điểm mấu chốt của cả mục. Một vòng bị bỏ lỡ và một vòng sai không đối xứng nhau, và khoảng cách giữa chúng rất lớn.

Một vòng bỏ lỡ để lại đúng phần trôi đã tích lũy. Phần đó có giới hạn, nó không lan ra chỗ khác, và một vòng sau vẫn có thể sửa nó.

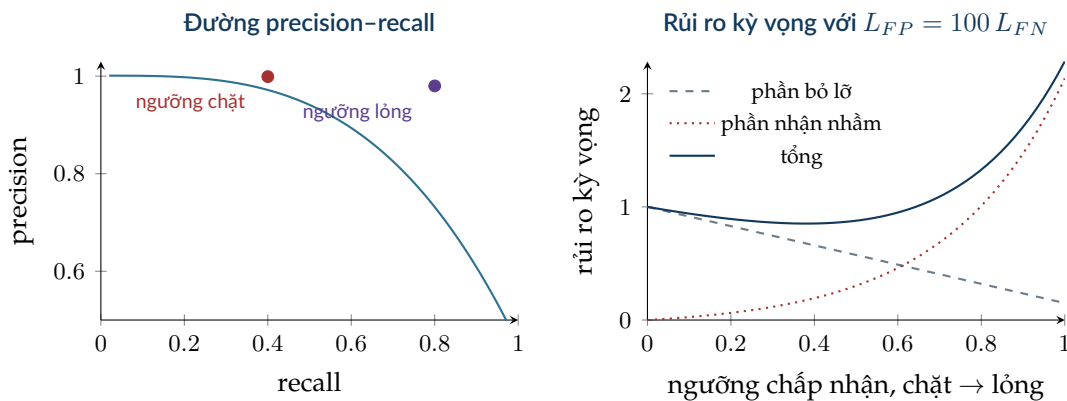
Một vòng sai thì khác hẳn. Tăng bốn nhận nó như một sự thật và phân bố lại sai số cho toàn bộ đồ thị. Bản đồ bị bẻ, và các khung hình khoá cách xa hàng trăm mét cũng bị dời. Hư hỏng không tự khỏi, và việc gỡ nó ra đòi cơ chế phục hồi bản đồ mà phần lớn hệ không có.

Viết thành rủi ro kỳ vọng thì rất gọn. Gọi p_{FP} là xác suất nhận nhầm và p_{FN} là xác suất bỏ lỡ, với L là thiệt hại tương ứng:

$$R = p_{FP} L_{FP} + p_{FN} L_{FN}.$$

Câu diễn giải: bạn không tối thiểu hoá số lần sai, bạn tối thiểu hoá *thiệt hại*. Khi L_{FP} lớn hơn L_{FN} chừng một bậc, điểm tối ưu dịch rất xa về phía precision. Đó là lý do cộng đồng nhận dạng địa điểm báo cáo “recall ở mức precision 100 %” thay vì recall trung bình.

Ví dụ tính tay được. Giả sử $L_{FP} = 100$ và $L_{FN} = 1$, tính bằng đơn vị tùy ý. Ngưỡng lỏng cho $p_{FN} = 0,2$ và $p_{FP} = 0,02$, nên $R = 2,0 + 0,2 = 2,2$. Ngưỡng chặt cho $p_{FN} = 0,6$ và $p_{FP} = 0,001$, nên $R = 0,1 + 0,6 = 0,7$. Ngưỡng chặt thắng, dù nó bỏ lỡ ba phần năm số vòng. Hình 1.8 vẽ cả hai đường.



Hình 1.8. Trái: đường precision–recall minh hoạ của một chuỗi cổng đóng vòng, kèm hai điểm vận hành. Phải: rủi ro kỳ vọng khi một vòng sai đắt gấp một trăm lần một vòng bỏ lỡ. Cực tiểu của đường tổng nằm lệch hẳn về phía ngưỡng chặt, dù ở đó recall chỉ khoảng bốn phần mười. Cả hai đồ thị là minh hoạ dạng hàm, không phải số đo.

E. Đo ở tầng hệ thống, và đọc đuôi phân phối

Recall và precision là chỉ số của bộ phát hiện. Chúng không nói bản đồ tốt lên hay xấu đi. Chỉ số hệ thống là sai số quỹ đạo, đo bằng *evo*, chạy nhiều lần.

Điểm kỹ thuật quan trọng: đọc *đuôi phân phối*, không đọc trung vị. Một vòng sai xảy ra hiếm, nên nó gần như không dời trung vị, nhưng mỗi lần xảy ra thì nó phá hỏng nguyên một lần chạy. Dấu hiệu đặc trưng vì thế rất dễ nhận: trung vị nhích lên một chút trong khi phân vị 95 nhảy lên nhiều

lần. Nếu chỉ nhìn trung vị, thay đổi trông như một khoản lỗ nhỏ đáng chấp nhận; nhìn đuôi thì nó là hổng. Đây là lý do một thay đổi ở tầng ba *bắt buộc* phải báo cáo cả hai con số.

Cách chọn số lần chạy và cách phân biệt cải tiến thật với nhiễu nằm ở B/07-chia-du-lieu-va-danh-gia và E/24-thi-nghiem-va-ablation. Ở đây chỉ cần một quy tắc: mọi so sánh về đóng vòng phải báo cáo cả trung vị lẫn phân vị 95, trên ít nhất mười lần chạy.

Còn một cạm bẫy về chân trị. Trên TUM-VI, chân trị chỉ phủ phòng có hệ bất chuyển động, nên các vòng đóng giữa hành lang không thể kiểm bằng chân trị. Ở những chuỗi như vậy, bằng chứng duy nhất là sai số quỹ đạo cuối cùng, không phải nhân đúng-sai của từng vòng.

F. Đo chi phí cho đúng: đuôi độ trễ và thời gian bị chặn

Còn một loại số phải đo, và nó không phải recall. Đóng vòng là mô-đun duy nhất trong hệ có hai chế độ chi phí rất khác nhau. Chi phí phát hiện chạy trên mọi khung hình khoá; chi phí hiệu chỉnh chỉ chạy khi có vòng.

Chi phí phát hiện phải đọc ở đuôi, không ở trung vị, và lý do là cấu trúc chứ không phải một con số cụ thể: số ứng viên mỗi khung hình khoá dao động rất mạnh, nên phân vị 99 của tầng này thường cách trung vị vài lần. Lập ngân sách theo trung vị nghĩa là trượt hạn đúng ở những chỗ hệ đang bận nhất — tức là đúng những chỗ bạn cần nó nhất.

Chi phí hiệu chỉnh thuộc loại khác hẳn, vì nó *chặn* luồng dựng bản đồ cục bộ. Nó chạy hiếm, nên nó gần như không xuất hiện trong bất kỳ thống kê trung bình nào; nhưng khi chạy thì nó dài hơn một chu kỳ khung hình rất nhiều, và trong suốt khoảng đó hệ không tạo được điểm bản đồ mới. Nếu robot đang di chuyển nhanh ngay lúc ấy, cửa sổ đó là một rủi ro mất bám vết thật. Con số phải đo là thời gian *bị chặn* của luồng dựng bản đồ, không phải thời gian chạy của hàm hiệu chỉnh.

Ba quy tắc đo rút ra được, và cả ba đều dễ bỏ sót.

1. Đo trên phần cứng đích khi hệ đang chạy đủ tải, không đo mô-đun đứng một mình.
2. Báo cáo phân vị 99 bên cạnh trung vị, cho cả phát hiện lẫn hiệu chỉnh.
3. Ghi lại thời gian mà các luồng khác bị chặn, vì đó mới là chi phí hệ thống thật của một vòng.

G. Ba dấu hiệu của một vòng sai mà bạn chưa phát hiện

Vòng sai hiếm khi tự khai. Nó không sinh ra lỗi, không làm sập hệ, và trên phần lớn chuỗi nó không dờn trung vị. Vì vậy cần biết ba dấu hiệu gián tiếp, và cả ba đều rẻ để ghi lại.

Dấu hiệu thứ nhất: bước nhảy tư thế tại thời điểm hiệu chỉnh. Sau khi đóng vòng, tư thế của khung hình khoá gần nhất bị dờn. Lượng dờn đó có ý nghĩa: nó chính là phần trôi mà vòng vừa xoá. Nếu nó lớn hơn nhiều so với trôi hợp lý của quãng đường đã đi, vòng đó đáng nghi. Ghi lại chuẩn của lượng dờn cho mỗi lần hiệu chỉnh là một dòng mã.

Dấu hiệu thứ hai: chi phí còn dư sau tối ưu đồ thị tăng vọt. Một ràng buộc sai mâu thuẫn với phần còn lại của đồ thị, nên nó để lại phần dư lớn ở chính nó hoặc ở các cạnh lân cận. Ghi tổng chi phí trước và sau mỗi lần tối ưu, rồi so với các lần trước, cho một cảnh báo sớm.

Dấu hiệu thứ ba: quỹ đạo có đoạn gấp khúc không giải thích được bằng chuyển động. Robot không dịch chuyển tức thời. Một đoạn quỹ đạo gãy góc tại đúng thời điểm hiệu chỉnh là bằng chứng hình học trực tiếp.

Ba dấu hiệu này đều là dấu hiệu *sau khi* vòng đã được nhận. Chúng không thay được công ở tầng ba, nhưng chúng biến một lỗi im lặng thành một lỗi có triệu chứng. Với một hệ chạy nhiều giờ

ngoài hiện trường, đó là khác biệt giữa việc phát hiện vấn đề trong ngày hôm đó và phát hiện nó sau ba tuần.

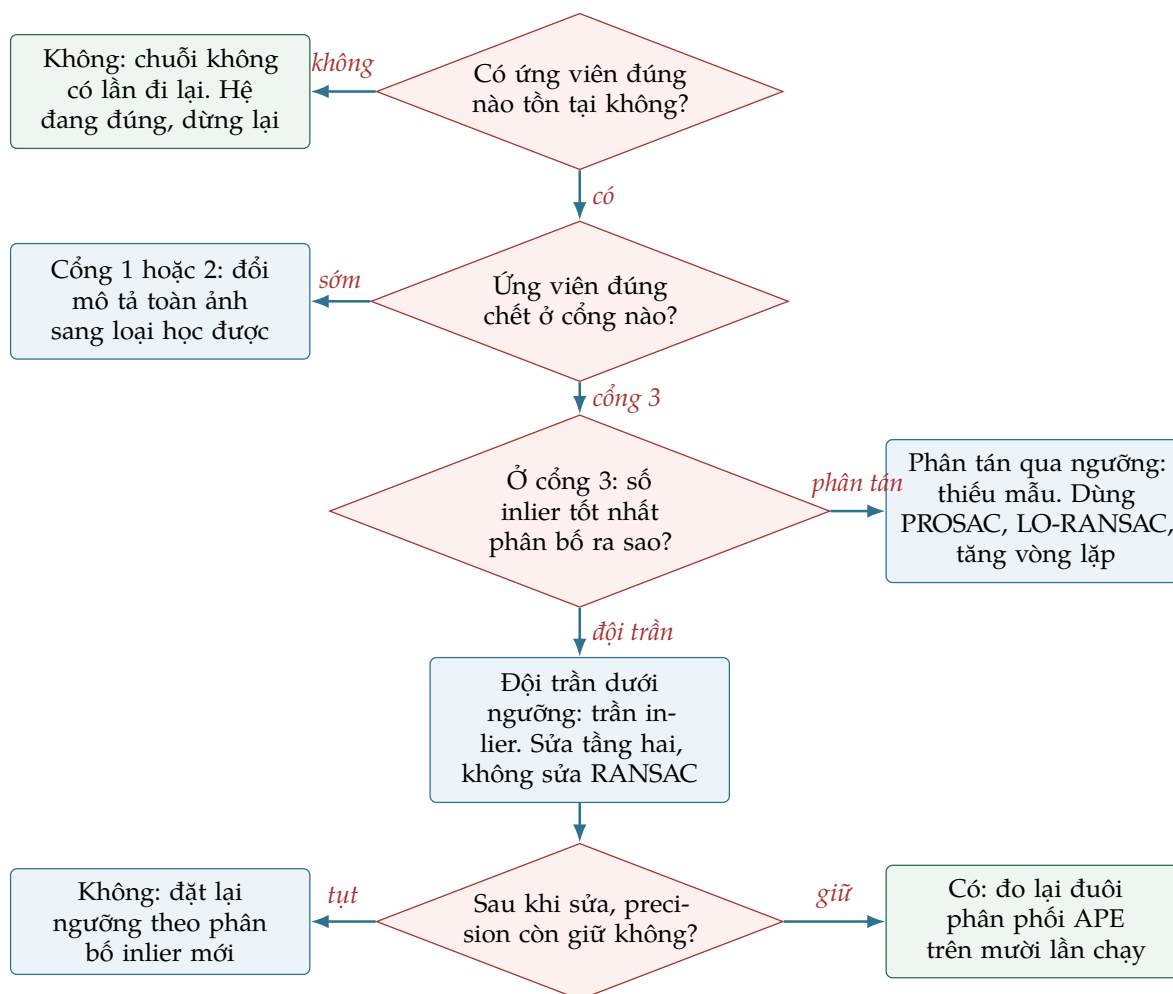
H. Hàng phòng thủ thứ hai: để bộ tối ưu tự chối vòng xấu

Vì precision không bao giờ đạt tuyệt đối, nên có một dòng nghiên cứu đặt lớp bảo vệ ở tầng bốn. Ý tưởng chung là để bộ tối ưu tự hạ trọng số hoặc tự tắt một ràng buộc trông không nhất quán với phần còn lại của đồ thị.

Ba cách kinh điển. Ràng buộc có công tắc gắn thêm một biến ẩn cho mỗi vòng, và biến đó được tối ưu cùng tư thế [32]. Co giãn hiệp phương sai động cho cùng hiệu ứng bằng một biểu thức đóng, rẻ hơn [33]. Mô hình hỗn hợp cực đại thay mỗi ràng buộc bằng một hỗn hợp Gauss, trong đó một thành phần rất rộng đóng vai trò “ràng buộc này có thể sai” [34]. Ngoài ra còn hướng kiểm tra tính nhất quán của cả nhóm vòng theo thời gian rồi loại cả nhóm nếu chúng mâu thuẫn nhau [35].

Điểm cần nói thẳng: các cơ chế này giảm thiệt hại, chúng không thay được một cổng kiểm chứng tốt. Chúng cũng có chi phí riêng, vì một ràng buộc đúng nằm ở vùng đuôi cũng có thể bị tắt nhầm. Với hệ đang chạy thời gian thực, hướng rẻ nhất vẫn là đặt ngưỡng chặt ở tầng ba và chấp nhận bỏ lỡ.

Hình 1.9 gói toàn bộ chương thành một cây quyết định chẩn đoán.



Hình 1.9. Cây chẩn đoán đóng vòng. Điểm quan trọng là thứ tự: hai câu hỏi đầu là phép đo, không phải lựa chọn kỹ thuật, nên bạn không được phép đi tiếp khi chưa có số. Giả thuyết làm việc — và chỉ là giả thuyết — là trên một hệ stereo-inertial chạy chuỗi ngắn cùng điều kiện ánh sáng, nhánh hay gặp nhất đi thẳng xuống ô “trần inlier”, tức phải sửa ở tầng ghép đặc trưng chứ không ở tầng RANSAC. Nếu hệ của bạn không đi vào nhánh đó, cái sai nằm ở giả thuyết chứ không ở cây.

1.7 Giới hạn và trường hợp hỏng

Giới hạn lớn nhất của cả chương phải nói thẳng ngay: chương này *không chứa số đo*. Nó không nói với bạn rằng cổng ba giết bao nhiêu phần trăm ứng viên trên hệ nào, vì một con số như thế chỉ có nghĩa kèm theo một hệ cụ thể, một nhóm tập dữ liệu cụ thể và một chính sách chọn khung hình khoá cụ thể — và khi thiếu ba thứ đó, nó gây hại nhiều hơn giúp. Thứ chương này cho bạn là bộ khung: bốn tầng, các cổng, phép đo đặt ở đâu, và cách đọc kết quả. Mọi phát biểu định tính trong chương — “cổng ba thường là chỗ lọc mạnh nhất”, “trần inlier hay gặp hơn thiếu mẫu trên hệ stereo-inertial chuỗi ngắn” — là giả thuyết làm việc, hợp lý về cơ chế nhưng chưa được chương này chứng minh. Hãy coi chúng là chỗ để bắt đầu tìm, không phải kết luận để trích dẫn.

Kèm theo đó là một giới hạn về phạm vi. Toàn bộ lập luận được viết cho một hệ stereo-inertial chạy chuỗi ngắn, trong nhà hoặc bán trong nhà, cùng điều kiện ánh sáng. Ba đặc điểm ấy quyết định kết luận.

Nếu đổi sang đơn ảnh, tỉ lệ trôi tỉ lệ sẽ lớn hơn, Sim3 thật sự cần bảy bậc, và tỉ lệ inlier thấp hơn. Nếu đổi sang ngoài trời qua nhiều buổi trong ngày, tầng một sẽ trở thành nút thắt thật, và kết luận “đừng thay DBow2” đảo ngược. Nếu đổi sang bản đồ hàng trăm nghìn khung hình khoá,

tf-idf bão hoà và truy hồi học được thắng rõ. Nói cách khác, kết luận trung tâm của chương là một kết luận *có điều kiện*, và điều kiện phải đi kèm mỗi lần nó được nhắc lại.

Chế độ hỏng thứ hai là chính phép đo. Bảng sống sót đo được các ứng viên *đã được đề xuất*. Nó mù hoàn toàn với các lần đi lại mà tăng một không đề xuất gì. Muốn thấy phần mù đó, phải dựng danh sách ứng viên bằng chân trị chứ không bằng bộ truy hồi, tức là chạy một thí nghiệm oracle. Không có bước đó, bạn có thể kết luận rằng tăng một tốt trong khi thật ra nó đang im lặng bỏ sót.

Chế độ hỏng thứ ba nằm ở chân trị. Ngưỡng khoảng cách để gọi hai khung hình là cùng nơi là một lựa chọn, không phải sự thật. Hai khung hình cách nhau ba mét nhưng quay lưng vào nhau thì không hề nhìn chung một cảnh. Một nhãn chỉ dựa trên khoảng cách sẽ gọi chúng là ứng viên đúng, và tăng ba loại chúng là hoàn toàn hợp lý. Nhãn tốt hơn phải tính cả phần chồng lấn trường nhìn, và việc bỏ qua điều này làm recall trông thấp hơn thực tế.

Chế độ hỏng thứ tư mang tính hệ thống. Khi vòng được chấp nhận, việc hiệu chỉnh bản đồ chặn luồng dựng bản đồ cục bộ trong một khoảng dài hơn hẳn một chu kỳ khung hình, và trên phần cứng nhúng thì khoảng đó còn dài hơn. Nghĩa là mỗi vòng đóng thành công cũng là một cửa sổ rủi ro cho việc mất bám vết và cho tranh chấp luồng. Một hệ đóng nhiều vòng hơn chưa chắc là một hệ tốt hơn, và đó là lý do chỉ số cuối cùng phải là sai số quỹ đạo chứ không phải số vòng.

Cuối cùng là một giới hạn về niềm tin. Toàn bộ chương giả định rằng bạn tin vào phép đo của chính mình. Cái bẫy biến đếm ở mục trước cho thấy niềm tin đó có thể sai suốt nhiều tuần mà không có triệu chứng nào. Cách duy nhất để bảo vệ là kiểm chéo mỗi bảng sống sót bằng một đường độc lập, chẳng hạn đếm lại từ nhật ký thô thay vì tin biến đếm.

1.8 Thực hành

Repo và tài nguyên. [cvgl/Hierarchical-Localization](#) là quy trình định vị nên đọc trước tiên, vì nó chính là cái phễu bốn tầng ở dạng mã nguồn. [dorian3d/DBow2](#) để đọc lại phần bạn đang dùng. [QVPR/Patch-NetVLAD](#) và [AnyLoc/AnyLoc](#) cho hai đầu của phổ mô tả toàn ảnh. [gmberton/CosPlace](#) và [amaralibey/MixVPR](#) nếu bạn muốn thử huấn luyện lại. [cvgl/LightGlue](#) cho ghép cặp có chi phí thích nghi, và [zju3dv/LoFTR](#) cho ghép cặp dày. [nianticlabs/ace](#) nếu tình huống triển khai của bạn là toà nhà cố định. [colmap/colmap](#) và [MichaelGrupp/evo](#) là hai công cụ hạ tầng bền nhất trong danh sách. Danh sách tên mô hình này chốt tại tháng 9/2026 và sẽ cũ trong sáu tới mười hai tháng; các repo hạ tầng như COLMAP và evo thì bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Bạn đặt biến đếm ở đâu để biết tầng nào đang giết vòng, và vì sao đặt trước câu lệnh `if` lại làm sai nhãn cột? Nhìn một biểu đồ số inlier tốt nhất, bạn phân biệt thiếu mẫu với trần inlier bằng dấu hiệu nào? Vì sao hồi quy tư thế trực tiếp lại nguy hiểm hơn hồi quy toạ độ cảnh trong một hệ SLAM? Nếu một vòng sai đất gấp một trăm lần một vòng bỏ lỡ, ngưỡng tối ưu nằm ở đâu trên đường precision-recall?

Mini project. Xem khối dự án ở cuối chương: dựng bảng sống sót bốn tầng cho chính hệ của bạn, kèm nhãn đúng-sai từ chân trị.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Bốn tầng của đóng vòng	Lấy ứng viên, ghép đặc trưng, kiểm chứng hình học, tối ưu tư thế. Mỗi tầng là một bộ lọc riêng, nên câu hỏi đúng luôn là “tầng nào đang hẹp” chứ không phải “hệ có đóng vòng không”.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Bảng sống sót từng cổng	Số ứng viên vượt qua mỗi cổng, tính theo phần trăm của số ứng viên tới cổng đó. Nó là phép đo rõ nhất cho biết tầng nào giết vòng, và dụng nó chỉ tồn vài chục dòng mã.
●●● Biến đếm đặt trước câu lệnh <code>if</code> — hậu quả?	Cột mang tên cổng đó thật ra đếm số ca đã vượt cổng trước nó, nên mọi nhân cột dịch một bậc và cổng bị đổ tội luôn là cổng đứng <i>sau</i> thủ phạm thật. Bảng vẫn nhất quán với chính nó nên không có triệu chứng nào. Cách phòng: tăng biến đếm bên trong khối <code>if</code> .
●●● Trần inlier	Số cặp đúng thật sự tồn tại đã ít hơn ngưỡng, nên không cách rút mẫu nào cứu được. Dấu hiệu nhận biết: số inlier tốt nhất của các ứng viên đúng đội trần ngay dưới ngưỡng.
●● Thiếu mẫu	Inlier có đủ nhưng số vòng lặp không đủ để rút trúng một tập mẫu toàn inlier. Dấu hiệu: số inlier tốt nhất phân tán qua ngưỡng. Chữa bằng PROSAC, LO-RANSAC, hoặc tăng số vòng lặp.
●● Công thức hội tụ của RANSAC	$p = 1 - (1 - w^m)^N$, với w là tỉ lệ inlier, m là cỡ tập mẫu tối thiểu, N là số vòng lặp. Thay thử với $m = 3$, $N = 300$: $w = 0,2$ cho $p \approx 0,91$, còn $w = 0,1$ chỉ cho $0,26$ — chia đôi w đủ để biến một cổng gần như luôn mở thành một cổng gần như luôn đóng.
●●● Ba giả định ẩn khi rút tập mẫu từ ba xuống hai	Một: inlier có đủ, chỉ thiếu lượt rút trúng — sai nếu biểu đồ mang Dạng B. Hai: trọng lực ghim hai bậc xoay <i>đúng</i> — sai nếu phần nghiêng còn dư đáng kể. Ba: thêm vòng thì bản đồ chỉ tốt lên — sai vì mỗi lần hiệu chỉnh là một cửa sổ rủi ro. Phép kiểm rõ nhất: tăng N lên mười lần trước đã.
●● Nhãn đúng–sai từ chân trị	Gắn nhãn cho từng ứng viên bằng khoảng cách tư thế thật, với một ngưỡng do bạn chọn — vài mét trong nhà, rộng hơn nhiều ngoài trời. Nó biến bảng sống sót thành hai bảng, và chỉ khi đó mới biết tầng hẹp đang loại đúng hay loại sai.
●●● Túi từ DBoW2	Biểu đồ tần suất các từ thị giác nhị phân, dùng lại chính mô tả ORB đã trích cho bám vết nên gần như miễn phí. Gây khi đổi ánh sáng mạnh, lệch góc nhìn lớn, cấu trúc lặp, hoặc nền ít vân.
●●● NetVLAD	Gộp phần dư quanh K tâm cụm, với phép gán làm mềm bằng softmax nên khả vi. Nhờ đó cả tâm cụm lẫn mạng trích đặc trưng đều học được từ ảnh cùng nơi khác thời điểm.
●● Patch-NetVLAD	Xếp hạng lại danh sách ngắn bằng các mảnh đặc trưng có tính tới vị trí không gian. Nó kéo một phần việc kiểm chứng lên tầng truy hồi, và chi phí tăng theo số ứng viên.
●● HF-Net	Một <i>backbone</i> chung, ba đầu ra: mô tả toàn ảnh, điểm đặc trưng, mô tả cục bộ. Bài học thiết kế là trả một lần cho một bức ảnh thay vì trả hai lần cho hai mạng.
●● CosPlace	Đổi huấn luyện nhận dạng địa điểm từ xếp hạng bộ ba sang phân lớp trên các ô địa lý. Nó cho thấy nút thắt khi ấy nằm ở quy trình dữ liệu chứ không ở kiến trúc.
●● SALAD	Xem việc gán đặc trưng vào cụm như bài toán vận tải tối ưu, kèm một cụm thùng rác cho đặc trưng vô ích, đặt trên nền DINOv2.
●● AnyLoc	Lấy đặc trưng trung gian của mô hình nền rồi gộp bằng VLAD, không huấn luyện lại. Mạnh nhất khi ra ngoài phân phối; đắt hơn về tính toán và mô tả dài hơn.
●●● Phân bác của Sattler 2019	Hồi quy tư thế trực tiếp hành xử gần với truy hồi ảnh có nội suy hơn là với định vị dựa trên cấu trúc. Nó nội suy tốt trong phân phối tư thế huấn luyện và ngoại suy rất tệ ngoài đó.
●● Hồi quy toạ độ cảnh	Mạng dự đoán toạ độ ba chiều cho từng điểm ảnh, rồi PnP trong RANSAC giải tư thế. Khác hồi quy tư thế ở chỗ nó giữ lại bộ giải hình học, nên vẫn có inlier để làm cổng.
●● ACE	Tách <i>backbone</i> không phụ thuộc cảnh khỏi một đầu MLP nhỏ riêng cảnh; bản đồ của một cảnh thu về đúng trọng số của đầu MLP đó. Không hợp vòng SLAM trực tuyến vì bản đồ ở đó lớn dần liên tục.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● hloc	Truy hồi, trích đặc trưng học được, ghép cặp học được, rồi PnP và RANSAC trên mô hình ba chiều. Chính là cái phễu bốn tầng với ba tầng đầu học được và tầng cuối giữ nguyên hình học.
●●● Vì sao điểm vận hành lệch về precision?	Vì một vòng sai bẽ cả bản đồ và không tự khỏi, còn một vòng bỏ lỡ chỉ để lại phần trôi có giới hạn. Khi thiệt hại lệch một bậc, cực tiểu rủi ro kỳ vọng dịch hẳn về ngưỡng chặt.
●● APE trung vị gần như không đổi nhưng phân vị 95 nhảy lên nhiều lần — nghĩa là gì?	Dấu hiệu điển hình của vòng sai: nó hiếm nên không dời trung vị, nhưng mỗi lần xảy ra thì phá một lần chạy. Luôn báo cáo cả trung vị lẫn đuôi trên ít nhất mười lần chạy.
●● Ràng buộc có công tắc	Gắn thêm một biến ẩn cho mỗi vòng để bộ tối ưu tự tắt ràng buộc không nhất quán. Nó giảm thiệt hại nhưng không thay được một công kiểm chứng tốt, và có thể tắt nhầm ràng buộc đúng.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [1]	Chuỗi bốn tầng ở Hình 1.1 là mô tả trực tiếp mô-đun đóng vòng và gộp bản đồ của nó	Hạ tầng tham chiếu; mã nguồn mở
hloc [8]	Thay ba tầng đầu bằng mô-đun học được, giữ nguyên bộ giải hình học ở tầng cuối	Quy trình chuẩn cho định vị trực quan
LightGlue [28]	Ghép cặp học được với chi phí thích nghi theo độ khó cặp ảnh — đúng chỗ trần inlier	Dùng rộng rãi, có bản ONNX
AnyLoc [14]	Đặc trưng mô hình nền cộng gộp VLAD, không huấn luyện lại	Nhắm bài toán ngoài phân phối
ACE [20]	Tách <i>backbone</i> không phụ thuộc cảnh khỏi đầu MLP riêng cảnh, nên dựng bản đồ là huấn luyện một đầu nhỏ	Hợp với triển khai một toà nhà cố định
Đo trên nhánh của bạn	Bảng sống sót từng cổng, nhãn chân trị, phép thử tăng N trước khi hạ m	Chương không đưa số; nó đưa chỗ đặt biến đếm

MINI PROJECT SLAM · Bảng sống sót bốn tầng cho bộ đóng vòng của bạn · 1–2 ngày

Mục tiêu	biến câu “vòng không nổ” thành bốn con số, rồi tách bốn con số ấy theo nhãn đúng–sai để biết tầng hẹp đang loại đúng hay loại sai.
Dữ liệu	bốn chuỗi có lần đi lại thật và một chuỗi đi thẳng không quay lại, làm đối chứng âm. Cấu hình mặc định, không đổi tham số nào.
Các bước	thêm một biến đếm <i>bên trong</i> mỗi khối if của chuỗi cổng, không đặt trước nó; ghi mỗi ứng viên thành một dòng nhật ký gồm mã khung hình khoá, số khớp túi từ, số inlier tốt nhất của RANSAC, và kết cục; chạy mười lần mỗi chuỗi; tra chân trị cho từng cặp khung hình khoá rồi gắn nhãn đúng nếu khoảng cách dưới bốn mét; lập bảng sống sót hai lần, một lần chung và một lần tách theo nhãn; cuối cùng vẽ biểu đồ số inlier tốt nhất cho hai nhóm nhãn.
Xong khi	bạn nói được câu “trên hệ của tôi, $X\%$ số ca chết nằm ở cổng Y , và trong đó $Z\%$ là loại đúng” kèm bảng số. Kiểm tra chéo bắt buộc: chuỗi đi thẳng phải cho gần như không có ứng viên đúng nào; nếu nó vẫn cho nhiều, nhãn chân trị của bạn sai. Kiểm tra thứ hai: đếm lại một cổng từ nhật ký thô và đối chiếu với biến đếm.
Nối với	E/24 cho thiết kế thí nghiệm và số lần chạy; F/28 là nơi bạn sẽ sửa nếu kết quả chỉ vào trần inlier; B/07 cho cách định nghĩa recall và precision trước khi báo cáo.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Đặt một biến đếm *bên trong* mỗi khối *if* của chuỗi cổng, cộng một dòng nhật ký cho mỗi ứng viên. Đặt trước câu lệnh *if* thì cột đếm “đã tới cổng” chứ không phải “đã qua cổng”, nên mọi nhân dịch một bậc và thủ phạm bị đổ nhầm.
- A2. Bằng nhãn chân trị. Tách bảng sống sót làm hai theo nhãn, rồi nhìn phân bố số inlier tốt nhất của riêng nhóm SAI: nếu nó tắt hẳn từ lâu trước ngưỡng thì mọi ca chết trong nhóm ấy là loại đúng, và một tỉ lệ sống sót thấp là bằng chứng cổng khỏe mạnh chứ không phải cổng hỏng.
- A3. Nếu nguyên nhân thật là trần inlier chứ không phải thiếu mẫu thì không cách rút mẫu nào chạm tới ngưỡng, vì số cặp nhất quán hình học vốn đã ít hơn ngưỡng. Dấu hiệu là Dạng B ở Hình 1.2, và phép kiểm rẻ nhất là tăng số vòng lặp lên mười lần: nếu tăng N không đổi gì thì hạ m cũng sẽ không đổi gì.
- A4. Vì nếu bảng sống sót cho thấy hai cổng đầu để lọt gần hết ứng viên thì tăng lấy ứng viên không phải nút thắt. Một bộ truy hồi tốt hơn khi ấy chỉ đưa thêm ứng viên vào một cái phiếu đang nghẽn ở tầng ba, nên số vòng đóng được không đổi dù recall theo số ứng viên tăng rõ rệt.
- A5. Vì thiết hại lệch. Một vòng sai bẻ toàn bộ đồ thị và không tự khỏi; một vòng bỏ lỡ chỉ để lại phần trôi có giới hạn. Với $L_{FP} = 100 L_{FN}$, ngưỡng chặt cho rủi ro 0,7 còn ngưỡng lỏng cho 2,2.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn đi bộ trong một thành phố lạ và muốn biết mình đã đi qua chỗ này chưa. Bạn làm bốn việc, theo thứ tự. Đầu tiên bạn liếc nhanh và thấy vài góc phố quen quen. Sau đó bạn nhìn kỹ hơn, tìm những chi tiết cụ thể: cái biển hiệu, vết nứt trên tường, chậu cây ở góc. Tiếp theo bạn thử xem những chi tiết ấy có nằm đúng vị trí tương đối với nhau như trong trí nhớ không. Nếu có, bạn kết luận đã đi qua đây, và bạn sửa lại toàn bộ bản đồ trong đầu.

Bây giờ giả sử bạn không bao giờ nhận ra chỗ cũ. Phản xạ thông thường là đổ lỗi cho cái liếc đầu tiên, rồi cố nhìn kỹ hơn ngay từ bước một. Nhưng nếu bạn ghi lại xem mình dừng ở bước nào, bạn sẽ thấy điều khác. Bạn nhận ra góc phố rất tốt. Bạn dừng ở bước thứ ba, vì bạn chỉ nhớ được ba chi tiết, mà ba chi tiết thì không đủ để chắc chắn. Vấn đề nằm ở bước thứ hai: bạn ghi nhớ quá ít chi tiết.

Cái giá của việc dễ tính hơn thì rất nặng. Nếu bạn nhận nhầm một hành lang giống hệt hành lang khác, bạn không chỉ sai ở chỗ đó. Bạn sẽ kéo lệch toàn bộ bản đồ trong đầu, kể cả những đoạn bạn nhớ hoàn toàn chính xác. Một lần nhận nhầm phá nhiều hơn mười lần không nhận ra. Vì vậy quy tắc đúng là: thà đi qua chỗ cũ mà không biết, còn hơn tin nhầm một chỗ mới là chỗ cũ.

Chỗ không rút gọn được: lý do vì sao “ba chi tiết là không đủ” lại là một con số cụ thể chứ không phải cảm tính. Nó đến từ một phép tính xác suất về việc rút ngẫu nhiên bao nhiêu lần thì trúng một bộ chi tiết đều đúng. Mọi cách nói đơn giản hơn đều làm mất chỗ quan trọng nhất: bạn phải phân biệt “rút chưa đủ nhiều lần” với “vốn dĩ không có đủ chi tiết đúng để mà rút”.

Một câu: Đóng vòng là bốn cái sàng nối tiếp, và việc đầu tiên phải làm không phải là đổi cái sàng nào, mà là đo xem cái nào đang giữ lại thứ bạn cần.

1.9 Kết nối

F/27-dat-hoc-sau-vao-dau là tiền đề trực tiếp: khung bốn cổng ở đó chính là thứ được áp dụng nguyên vẹn trong mục quyết định có thay DBow2 hay không. F/28-dac-trung-va-ghep-cap là chương anh em và là chỗ mà giả thuyết trung tâm của chương này chỉ tới: nếu biểu đồ inlier của bạn mang Dạng B thì việc phải sửa nằm ở đó. B/07-chia-du-lieu-va-danh-gia cho định nghĩa recall và precision cùng cách chia dữ liệu để những con số ấy có nghĩa. E/24-thi-nghiem-va-ablation cho cách quyết định bao nhiêu lần chạy là đủ, và cho khái niệm thí nghiệm oracle mà mục đo dùng lại nguyên vẹn. E/25-do-tin-cay-va-van-hanh là nơi lập luận rủi ro kỳ vọng được trình bày tổng quát, còn ở đây nó chỉ là một trường hợp riêng với thiết hại rất lệch. F/34-imu-bat-dinh-va-trien-khai nối tiếp về hai chuyện: phương trọng lực từ

IMU đã được dùng ở đây để rút bậc tự do, và ngân sách độ trễ trên Orin quyết định mọi lựa chọn trong bảng so sánh.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một tỉ lệ sống sót rất thấp ở tầng kiểm chứng không tự nó chứng minh rằng tầng đó hỏng, và bạn cần thêm phép đo nào?
2. Nếu biểu đồ số inlier tốt nhất của các ứng viên đúng đội trần ngay dưới ngưỡng, ba hướng chữa nào bị loại ngay lập tức, và vì sao?
3. Lý thuyết RANSAC hứa hẹn một bước nhảy lớn về xác suất hội tụ khi rút tập mẫu từ ba xuống hai. Ba giả định ẩn của lời hứa đó là gì, và với mỗi giả định, phép đo rẻ nhất để biết nó có đúng trên hệ của bạn là gì?
4. Bạn thay DBoW2 bằng một mô tả học được và recall theo số ứng viên tăng rõ rệt, nhưng số vòng đóng được không đổi. Chuyện gì đang xảy ra, và bạn kiểm bằng cách nào?
5. Vì sao hồi quy tư thế trực tiếp lại nguy hiểm hơn hồi quy toạ độ cảnh khi đưa vào một hệ SLAM đang chạy, ngoài chuyện sai số lớn hơn?
6. Đưa một bộ ghép cặp học được vào tầng hai làm tăng số cặp. Vì sao việc đó buộc bạn phải đo lại ngưỡng ở tầng ba, thay vì giữ nguyên?
7. Trên một chuỗi mà chân trị chỉ phủ một phần đường đi, bạn dựa vào bằng chứng nào để nói một vòng vừa đóng là đúng hay sai?
8. Vì sao một hệ đóng nhiều vòng hơn chưa chắc là một hệ tốt hơn, kể cả khi mọi vòng đều đúng?

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] D. Gálvez-López and J. D. Tardós, “Bags of Binary Words for Fast Place Recognition in Image Sequences,” *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [3] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [4] B. K. P. Horn, “Closed-form solution of absolute orientation using unit quaternions,” *Journal of the Optical Society of America A*, vol. 4, no. 4, pp. 629–642, 1987.
- [5] S. Umeyama, “Least-squares estimation of transformation parameters between two point patterns,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.
- [6] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [7] S. Hausler, S. Garg, M. Xu, et al., “Patch-NetVLAD: Multi-Scale Fusion of Locally-Global Descriptors for Place Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [8] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., “From Coarse to Fine: Robust Hierarchical Localization at Large Scale,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [9] G. Berton, C. Masone, and B. Caputo, “Rethinking Visual Geo-localization for Large-Scale Applications,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [10] A. Ali-bey, B. Chaib-draa, and P. Giguère, “MixVPR: Feature Mixing for Visual Place Recognition,” in *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2023.
- [11] G. Berton, G. Trivigno, B. Caputo, et al., “EigenPlaces: Training Viewpoint Robust Models for Visual Place Recognition,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [12] S. Izquierdo and J. Civera, “Optimal Transport Aggregation for Visual Place Recognition,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [13] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [14] N. Keetha, A. Mishra, J. Karhade, et al., “AnyLoc: Towards Universal Visual Place Recognition,” *IEEE Robotics and Automation Letters*, vol. 9, no. 2, pp. 1286–1293, 2024.
- [15] A. Kendall, M. Grimes, and R. Cipolla, “PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization,” in *IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [16] T. Sattler, Q. Zhou, M. Pollefeys, et al., “Understanding the Limitations of CNN-based Absolute Camera Pose Regression,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

- [17] E. Brachmann, A. Krull, S. Nowozin, et al., “DSAC — Differentiable RANSAC for Camera Localization,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [18] E. Brachmann and C. Rother, “Learning Less is More — 6D Camera Localization via 3D Surface Regression,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [19] E. Brachmann and C. Rother, “Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2021.
- [20] E. Brachmann, T. Cavallari, and V. A. Prisacariu, “Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [21] E. Brachmann, J. Wynn, S. Chen, et al., “Scene Coordinate Reconstruction: Posing of Image Collections via Incremental Learning of a Relocalizer,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [22] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *CVPR*, 2016.
- [23] H. Strasdat, J. M. M. Montiel, and A. J. Davison, “Scale Drift-Aware Large Scale Monocular SLAM,” in *Robotics: Science and Systems (RSS)*, 2010.
- [24] O. Chum and J. Matas, “Matching with PROSAC — Progressive Sample Consensus,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005.
- [25] O. Chum, J. Matas, and J. Kittler, “Locally Optimized RANSAC,” in *DAGM Symposium on Pattern Recognition*, 2003.
- [26] D. Barath, J. Noskova, M. Ivashechkin, et al., “MAGSAC++, a Fast, Reliable and Accurate Robust Estimator,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [27] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [28] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [29] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [30] J. Edstedt, Q. Sun, G. Bökman, et al., “RoMa: Robust Dense Feature Matching,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [31] S. Lowry, N. Sünderhauf, P. Newman, et al., “Visual Place Recognition: A Survey,” *IEEE Transactions on Robotics*, vol. 32, no. 1, pp. 1–19, 2016.
- [32] N. Sünderhauf and P. Protzel, “Switchable Constraints for Robust Pose Graph SLAM,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [33] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., “Robust Map Optimization Using Dynamic Covariance Scaling,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2013.
- [34] E. Olson and P. Agarwal, “Inference on Networks of Mixtures for Robust Robot Mapping,” *The International Journal of Robotics Research*, vol. 32, no. 7, pp. 826–840, 2013.
- [35] Y. Latif, C. Cadena, and J. Neira, “Robust Loop Closing over Time for Pose Graph SLAM,” *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013.